Review

Vernon Kok, Absalom E. Ezugwu* and Micheal Olusanya

Reinforcement learning for mobile robot mapless navigation: a review of recent advances

https://doi.org/10.1515/jisys-2024-0547
Received December 30, 2024; accepted October 16, 2025; published online January 16, 2026

Abstract: Mobile robot navigation remains a critical challenge in robotics, with applications spanning autonomous vehicles, search and rescue, and other dynamic environments. In recent years, reinforcement learning (RL) has become a powerful approach for solving complex tasks such as robotic manipulation, gameplay, and autonomous driving. By enabling robots to learn optimal navigation strategies through interaction with their environment, RL offers a promising pathway to autonomous mobility. This review presents a comprehensive overview of recent advancements in RL as applied to mobile robot navigation. We begin by outlining core RL concepts, agents, environments, rewards, and value functions, explaining their roles in navigation. Key RL techniques, including Q-learning, deep reinforcement learning, Markov Decision Processes (MDPs), and policy gradient methods, are examined to highlight their transformative impact on navigation performance. The review also explores a range of practical applications and identifies current challenges and open research directions. Critical issues such as safety, sample efficiency, and scalability to real-world scenarios are discussed in depth to ensure robust deployment of RL-based systems. Lastly, this review synthesizes the state-of-the-art in reinforcement learning for mobile robot navigation, offering readers both a foundational understanding and valuable insights into emerging trends and future opportunities in this rapidly evolving field.

Keywords: reinforcement learning; mobile robot; dynamic environment; navigation; path planning; agent

1 Introduction

A fundamental capability of an intelligent agent is the ability to reason and make decisions. In nature, when an animal's surroundings are no longer favorable, it moves to a new environment. As human beings, we learn to navigate within our first few years on Earth. The human brain is so advanced that once we have acquired the skill, with practice, it can be achieved without even thinking about it; when you want to be in the kitchen, your limbs, in combination with your brain, handle the transition from A to B without you being aware of what is happening. The ability to safely navigate to a destination is a fundamental skill for survival [1]. Since time immemorial, we have been interested in improving our environment, be it through the development of tools, the discovery of fire to keep us warm, the development of carriages to carry loads, the development of trains to carry larger loads further, and recently the development of digital computing.

Compared to the human brain, the modern computer is a crude approximation. We do not yet have the computational power and models to express the complexity of the organic brain. Artificial intelligence (AI) is the

***Corresponding author: Absalom E. Ezugwu**, Unit for Data Science and Computing, North-West University, 11 Hoffman Street, Potchefstroom, 2520, South Africa, E-mail: absalom.ezugwu@nwu.ac.za. https://orcid.org/0000-0002-3721-3400
Vernon Kok, Unit for Data Science and Computing, North-West University, 11 Hoffman Street, Potchefstroom, 2520, South Africa, E-mail: 49942395@mynwu.ac.za
Micheal Olusanya, Department of Computer Science and Information Technology, Sol Plaatje University, Kimberley 8300, South Africa, E-mail: michael.olusanya@spu.ac.za

field of study that is concerned with answering questions regarding the nature of thinking and reasoning and how these can be translated into machines. The study of AI is primarily concerned with the development of autonomous agents that interact with their environment [2]. Mnih et al. [3] defined AI as the set of tools and techniques proposed to solve problems that require intelligence. A robot is the physical manifestation of an agent. An agent is a mathematical model learned through data, hand-designed rules, or trial and error. The agent must gather information, formulate some meaningful representation of the information received, and, in most cases, perform actions.

Navigation is a fundamental capability of any agent operating in the real world [4]. To gather information about our environment, we need to navigate and explore while simultaneously avoiding hazards that may be present. The primary goal of autonomous navigation is intervention-free movement from an initial starting point to a goal point [5]. If a map is available, one may have a target and a destination; the map allows one to visualize how to get from where we started to where we want to go. The map may also provide a view of where obstacles may be located in the case of a static environment. Navigation is a task that is relatively easy for humans, even though there is no concise formula to navigate without hurting oneself, mammalian brains combine internal representations with information from the environment to localize itself in its environment [6]. AI studies methods and techniques to mimic human intelligence in artificial systems. Computers permeate our everyday lives and can be found in multiple disciplines. In medicine, agents are applied to diagnose diseases, prescribe medicine and treatments, and analyze X-rays. In banking, AI can be applied to fraud detection, customer trend analysis, and financial forecast models. In the military, AI has been applied to unmanned aerial vehicles (UAVs) and search and rescue robots.

Traditional navigation methods rely on a predefined map of the environment to guide agents toward a goal or predefined rules to encode desired behavior [7]. In contrast, goal-driven navigation focuses on reaching a target state or location while dynamically avoiding obstacles, often without prior knowledge of the environment. With the increasing complexity of high-dimensional environments, reinforcement learning (RL) has emerged as a promising solution for control tasks, offering end-to-end policies that directly map raw sensory inputs to control actions [2], [8]. RL enables agents to adapt to previously unseen environments, improving real-time decision-making and generalization through interaction [9], [10]. Although classical approaches provide a strong foundation for current methods, they often require extensive feature engineering and environment modeling. In contrast, mapless navigation, which avoids the need for explicit map construction, has gained traction for its scalability and efficiency in real-world applications [1], [4], [11].

Designing an RL agent capable of navigating toward a goal while avoiding obstacles is a non-trivial challenge. The strength of RL lies in its ability to learn robust, generalized behaviors through interaction, with minimal reliance on handcrafted features. In this work, we aim to explore strategies for end-to-end learning of goal-driven, mapless navigation policies. Specifically, we review and analyze recent research efforts that address the problem of navigating from an arbitrary start state to a specified goal without prior environmental maps. This paper is structured around the following research questions.

- What are the emerging trends in robotic control using reinforcement learning
- Which reward functions are used to achieve mapless navigation?
- Which approaches have been used to modify an agent's behaviour in a given environment?
- What are the most frequently used state and action spaces?

1.1 Problem statement

Traditional navigation approaches have relied on the construction of environment maps, localization, and path-planning [6], [12]. The use of a predefined map of the environment as an initial step implies that map generation and maintenance must be repeated with each new environment. Recent state-of-the-art approaches have focused on enhancing state representation to be as informative as possible; however, it remains a challenge to select the most efficient setup to enable an agent to locate a goal in an environment while avoiding obstacles. The problem becomes worse when we consider the multitude of ways in which one can set state and action spaces.

In the absence of any environmental information, assuming that a sparse reward is natural and simplifies the agents design [13]. The design and implementation of suitable goal-driven reward functions is paramount to the success of an agent tasked with navigating an unknown environment. In most cases, the reward function is sparse; the agent only receives the reward at the end of the episode or upon collision with obstacles. When the reward function is over-engineered, it becomes hard to understand, modify, and interpret however sparse rewards make learning effective policies expensive [14]. The reward function of an agent is meant to encode the desired behavior of an agent, i.e., 'avoid obstacles' or 'move towards a goal'; however, there is no universal reward function available. This suggests that discovering a meaningful reward function requires many trial-and-error approaches. With the myriad of reward mechanisms available, the identification of which reward functions allow for goal-driven navigation of an agent becomes nontrivial.

Using the goal information in the reward mechanism implies that it is available at each timestep, which may not be available in resource-constrained applications. Methods are required to infer rewards directly from the observed states or autonomously identify sub-goals that reduce reward sparsity and increase environment exploration. The performance of a mapless navigation agent will be poor without the correct state definition, action space description, and reward mechanism. Correctly identifying which state space is applicable in which scenarios is the key to the success of an agent's performance. Furthermore, identifying which action definitions allow for the desired level of control is another factor to consider. Our aim with this review is to expose the reader to all the facets of navigation using reinforcement learning techniques, which enables well-informed agent designs that leverage state-of-the-art approaches that have proven successful.

1.2 Purpose and scope of review paper

This paper investigates how a reward mechanism can be designed to guide an agent toward achieving desired behaviors in an RL setting. Specifically, we focus on reward functions that rely on minimal assumptions about the agent's environment, aiming for generalizability and adaptability across different scenarios. Our study emphasizes approaches that use LiDAR and image data for state representation, and explores the action spaces that enable mapless navigation of mobile agents.

We conduct a comprehensive review of the key components involved in the design of RL agents for mapless navigation. This includes recent developments in state-space formulation, the design of reward functions, and the types of actions available to agents. The objective is to identify effective configurations that enable an agent to navigate from an arbitrary starting point to a predefined goal without relying on a map. To support our review, we draw from open-access literature available through sources such as Scopus, Web of Science, IEEE Xplore, SpringerLink, ScienceDirect, ACM Digital Library, and JSTOR. We aim to provide readers with an in-depth understanding of current strategies used in mapless navigation, with a particular focus on methods that utilize LiDAR and image data, either independently or in hybrid combinations. This review not only highlights state-of-the-art techniques, but also outlines key considerations for designing RL agents for control tasks in unstructured environments.

1.3 Research gap and justification for this study

Traditional navigation approaches in robotics have typically relied on hard-coded rules, extensive pretraining on large datasets, or precise mathematical models of both the agent and its environment. While these methods can be effective in constrained or well-defined settings, they suffer from poor generalization [14]. Specifically, agents designed using such approaches often fail when placed in environments that differ from those for which they were explicitly trained or programmed. For example, an agent trained or hard-coded to navigate a kitchen environment may struggle or completely fail when deployed in an outdoor setting. Moreover, collecting large, labeled datasets for training rule-based or sensing-and-avoidance agents becomes increasingly infeasible in unknown or dynamic environments. Similarly, hard-coded rules only apply to specific scenarios and do not scale

well to new contexts. This limitation significantly reduces the adaptability of traditional navigation systems in real-world, unstructured environments.

RL has emerged as a compelling alternative for control tasks in modern robotics due to its ability to learn behaviors through trial and error. Rather than explicitly programming rules, RL allows the agent to interact with the environment and autonomously discover policies that maximize cumulative reward. This makes it especially suitable for tasks like navigation, where specifying every contingency manually is impractical. However, the performance of an RL agent is highly dependent on three critical components: the state space, the reward function, and the action space. The ability to obtain information from the environment while avoiding irrelevant features is an essential task for any agent while the selection of what forms the state of an agent remains a trial-and-error process [15]. While numerous reward functions have been proposed for RL tasks, many rely on simplifying assumptions that are not feasible in real-world mapless navigation scenarios, such as assuming the agent always knows the exact distance to the goal. Currently, there is a lack of comprehensive studies that evaluate which reward functions are most suitable for real-world mapless navigation tasks and how they can be effectively implemented.

To bridge this gap, this review dissects recent reinforcement learning-based approaches for mapless navigation by focusing on their core components: state representation, reward structure, and action formulation. The goal is to identify which configurations of these components enable an agent to navigate autonomously from an arbitrary start point to a goal without relying on prior maps. In particular, we analyze state-of-the-art methods that utilize LiDAR and image data, either independently or in combination, as input to the agent's policy. LiDAR provides a cost-effective way to perceive the distribution of free space around the agent, while images offer rich semantic context. Approaches using LiDAR-based inputs are summarized in Table 2, image-based inputs in Table 3, and hybrid methods that combine both in Table 4. For each approach, we detail the variables that comprise the input space, the reward structures used to incentivize behavior (e.g., reaching goals or avoiding obstacles), and the formulation of the action space that enables effective navigation. The justification for this study lies in its comprehensive synthesis and analysis of recent RL approaches for mapless navigation. By distilling the key design choices regarding state, reward, and action components, this review provides valuable insights into how these elements interconnect to support robust, real-world agent behavior. In particular, the study serves as a practical guide for researchers designing reinforcement learning agents for autonomous navigation in complex and dynamic environments. Moreover, the technical contributions of this review work are summarized as follows.

- We examine how traditional data-driven methods have been applied to enable mapless control of mobile agents.
- We analyze how reinforcement learning has been used to automate key aspects of conventional agent control.
- We review recent advancements in navigation techniques for both known and unknown environments.
- We investigate the types of information that are most beneficial to agents and how this information should be processed effectively.

Figure 1 depicts the review structure, highlighting the principal components of the literature that collectively support and frame this study.

The remainder of the paper is structured as follows. In Section 2, we introduce state spaces, action spaces, and reward functions. Section 3 summarizes what mobile robotics is and highlights traditional approaches. Reinforcement learning and some of the most fundamental approaches to learning from interaction are introduced. Section 4 presents an overview of the research methodology used to conduct the review, and subsequently, a detailed discussion of the existing body of related work on the topic at hand is provided in Section 5. Furthermore, we present a general discussion based on the insights from the considerations that need to be taken when designing agents to achieve navigation in an environment without any knowledge of the environment *a priori*. Section 6 concludes the review work.

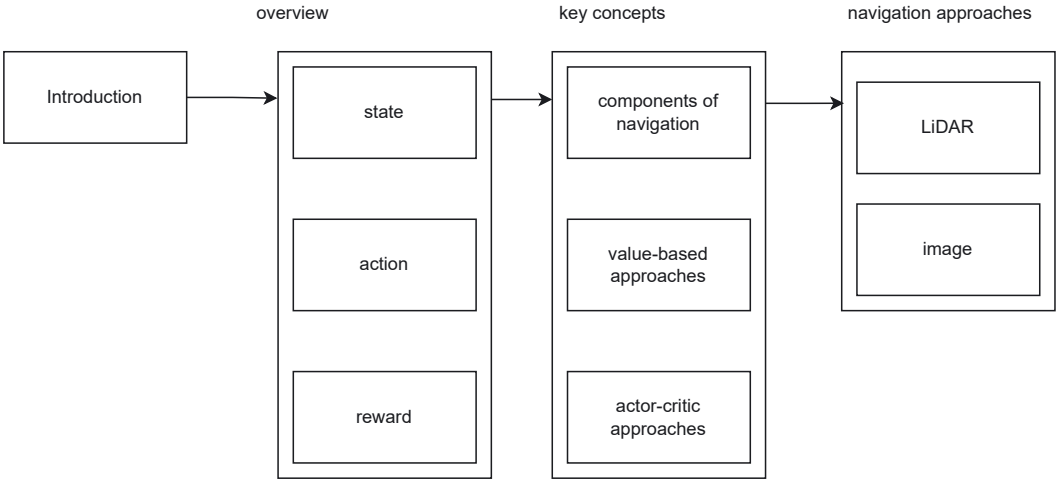

Figure 1: Structure of this review. Figure created by authors.

2 Overview of mobile robot navigation

A robot is a physical manifestation of an intelligent agent that reasons and makes decisions based on information received from its environment [16]. The robot is defined by the task that it needs to achieve; mobile robots are a subset of robots designed for movement. Robots can move on land, sea, and air; hybrid approaches allow robots to transition from air to sea. This movement is achieved through motors attached to wheels. An intelligent agent gathers information from its environment and processes the information gathered using its internal mechanisms or functions. In mobile robot navigation, the agent is equipped with sensors to perceive the state of its environment. The types and configurations of the sensors attached to the robot are specific to the task. Mobile robots are equipped with various sensors, and the various readings are combined to provide an informative representation of the state [17]. Mobile robots are defined by their ability to move around in their environment. A human may control the agent, also known as teleoperation, or the agent may navigate its environment autonomously. Mobile robots can be further classified according to how they achieve locomotion [18]. Figure 2 provides a taxonomy of mobile robots.

Navigation in an environment consists of perception, cognition, and control. Navigation is a fundamental ability of any intelligent system [19]. Perception refers to how the robot perceives its environment. A robot is given a 'peak' at its environment through the observed sensory measurements. Autonomous navigation allows for the exploration of environments that we cannot reach affordably: exploring a mine, navigating on other planets, and searching for a bomb in a room. Robots designed to be operated by humans, such as a crane or robot arm, are useful as tools; these robots cannot achieve the task autonomously.

In mobile robot navigation, the agent is equipped with sensors to perceive the state of its environment. The types and configurations of the sensors are specific to the task at hand. A robot may be equipped with motors to enable navigation in its environment. A mobile robot faces the challenge of localizing itself in the environment, planning a safe path to its goal, avoiding contact with obstacles, and navigating from one state to another. In the case of mapless navigation, the robot can only achieve local navigation, as global navigation would require full knowledge of the Markov Decision Process (MDP). Localization in the environment is an important task that a robot faces, which is achieved through inertial sensors. During design, one may include sensors; the sensors provide information to the agent that the designer considers useful for obstacle avoidance or goal-reaching. To localize itself in an environment, the robot may also be equipped with localization modules and may obtain information regarding its goal using GPS.

As discussed later, the agent may have a visual representation of the state obtained from a camera, such as visual navigation. The robot may receive a visual representation of its goal in visual navigation. The robot gains

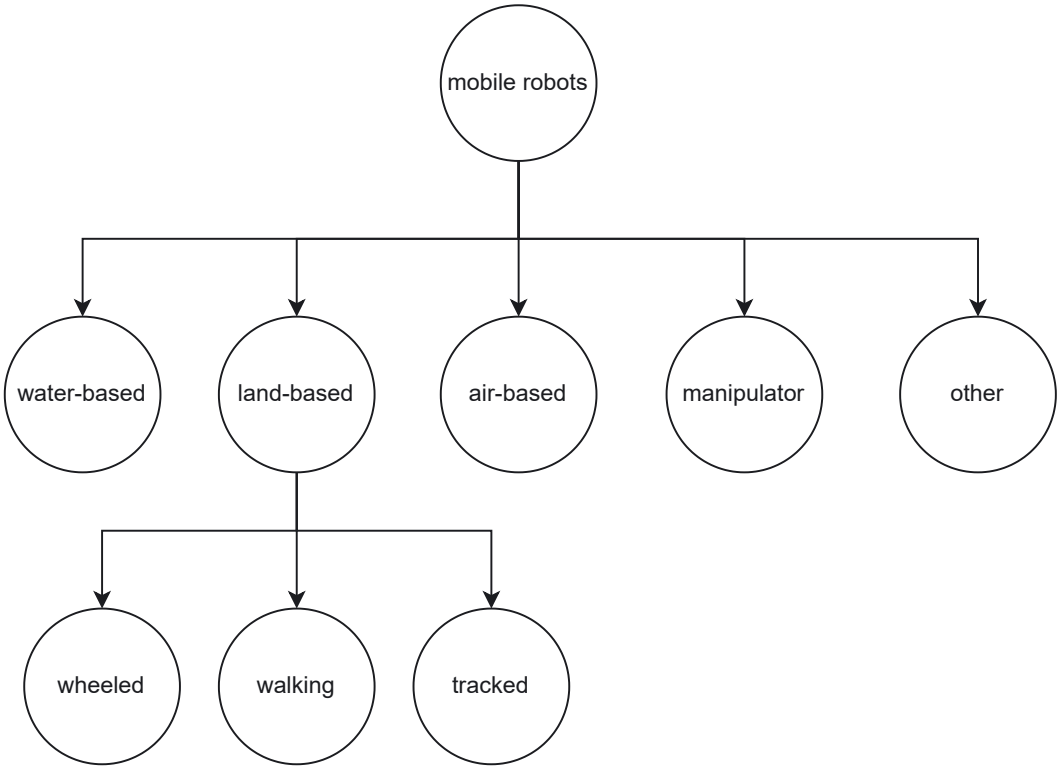

Figure 2: Mobile robot taxonomy. Figure created by authors.

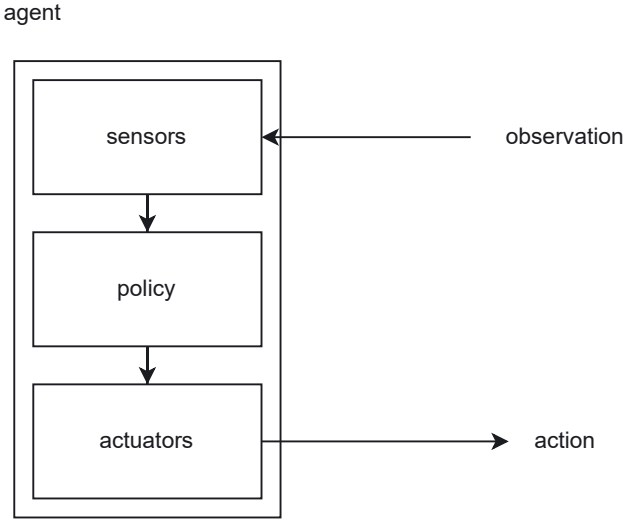

Figure 3: Graphical depiction of how an agent processes states to perform actions. Figure created by authors.

an understanding of the distribution of free space using LiDAR or laser range-finders, i.e. when the readings are large in a particular direction. The robot is then tasked with performing the correct actions to achieve the goal

state; in the visual navigation setting, this is the state from which the agent has its goal in its current frame. Figure 3 illustrates how information obtained from sensors is processed to produce an action.

2.1 Decision space

The set of all actions available to a robot in the action space. The power of a robot is its ability to perform actions without the need for human supervision. The actions must, therefore, be carefully chosen to reflect the nature of the task at hand, (e.g.) A robot that needs to open doors must know door dynamics and must be able to perform tasks like 'open door' and 'close door'. The action space can be either continuous or discrete, meaning that the actions can be continuous values that indicate linear and angular velocities or discrete actions like 'move left', 'move right', and 'move forward'. RL methods require large amount of trial-and-error to propogate information about the returns, thus smaller discrete action spaces are preffered [20]. In cases where a trained policy must be transferred to a real robot, keeping the actions available to the robot as close to those available to the real robot eases sim-to-real transfer [21]. The action space chosen has an impact on the final navigation, e.g., continuous actions produce smooth navigation trajectories, whereas discrete actions produce jerky movements. The choice of action space also determines which algorithm is used, although it is possible to discretize the actions and apply discrete algorithms to a continuous action space.

2.2 State space

The state space of a mobile robot is how the robot 'sees' the world. It can be an image frame from an onboard camera, a vector of distance readings, or a vector of linear and angular velocity. Good-quality data is essential to the performance of an agent. The data made available to an agent is a key factor in determining its success or failure [12]. The state is a product of the observation; the observation refers to the raw data obtained from the sensors, whereas the state refers to some meaningful preprocessed observation version. However, a naive approach would be to pack as much information as possible into an agent's state, more so, compact representations have been shown to improve learning ability [21]. Furthermore, a compact state representation reduces the complexity of the agents' design, which in turn reduces resource requirements.

The robot may measure its state or the state of its environment. Sensor fusion techniques combine information from multiple sensors to provide a holistic view of the robot's state. Sensors can provide a state representation, but may be noisy, as all sensors are subject to a certain noise level. The noisy nature of sensors may prove useful in some cases, as it introduces some element of randomness, which will help the agent generalize to unseen real-world environments where the agent may encounter data not seen in simulation [8]. Sensor fusion helps reduce the uncertainty of measurements obtained from one sensor. The Kalman Filter and Particle Filter are techniques that estimate the probability density function (PDF) from sensory readings. Each source of information has its drawbacks. LiDAR data itself is not descriptive enough, and RGB data makes sim-to-real transfer challenging as the simulators themselves are resource-intensive and may not account for all phenomena encountered in the real world. The sim-to-real transfer problem can be remedied by abstraction or learning a low-dimensional representation of the input data such that what the agent encounters in simulation is not that different from what it will encounter in real-world settings.

To help make navigation decisions, information from various data sources can be combined [22]. In resource-constrained environments sparse data representations may offer a cheaper alternative to high dimensional state representations at the cost of being less expressive and informative. The curse of dimensionality results in higher-dimensional states requiring more powerful models, larger datasets or more interactions, and powerful hardware to train these agents in a timely fashion. The simulation environment used is an attempt to replicate the real world; as such, the sensors used in the simulation are the ideal version of their real-world counterpart. Real-world sensors are noisy, and when selecting a state representation, it is important to have this in mind, as the agent's success is closely tied to the information it has available to make decisions. An agent maps states to actions. In

dynamic environments where obstacles or humans are moving, the state representation used may not be as effective as when there are static entities present. For example, when entities move, an agent may need several consecutive camera frames to capture the movement of the entity.

2.3 Goal specification

Goal specification in mobile robot navigation is critical to determining where a robot should go and what tasks it should accomplish within its immediate environment. Moreover, this type of specification helps the robot understand its mission and plan a path to reach its desired destination. Several studies have also shown that there are multiple ways in which agents' goals could be specified in mobile robot navigation, and the choice depends on the robot's capabilities, the environment, and the specific tasks at hand.

Among the common methods often adopted by researchers for goal specifications in mobile robot navigation are relative coordinates [23] that define the goal state relative to the robot's current position. For example, this process can involve instructing the robot to move three meters forward, turn 90° to the left, or go to a point two meters to the right of its current position. Also, because the relative coordinates method does not require prior knowledge of the global map, it simplifies the goal specification. Another similar method is the absolute coordinates [24], which involves specifying the robot's destination using absolute coordinates in the global reference frame. For example, one can provide specific coordinates (x, y) within a map or, say, a grid for which the robot is then expected to use its sensor devices and its localization algorithms to determine its current position and plan a path to the specified coordinates.

Other exciting approaches include landmarks or beacons [25], which involve specifying goals using specific identifiable landmarks or beacons within the agent's current environment. We also have the semantic description [26], where the agent is trained to understand some natural language or symbolic instructions. In scenarios involving voice-controlled robots [27], voice commands are often commonly used in which users can verbally command robots to perform certain tasks, such as moving to specific locations. Sensor-based goals are common in cases where the detection of objects or features in an environment is feasible. For example, a vacuum-cleaning robot can be programmed to move toward dirt or obstacles detected with its sensors. However, it is equally important to mention that the choice of the goal specification method depends on the complexity of the task, the sensors and perception capabilities of the robots, the level of autonomy required, and the user interface. More so, there are instances where combining all these methods can provide high-level flexibility and robustness in mobile robot navigation systems.

2.4 Reward function design

Designing reward functions within the context of mobile robot navigation is a crucial aspect of reinforcement learning and related optimization problems [28]. The reward function specifies a mobile robot's primary goal or objective, which invariably provides feedback on how well the robot performs. Several vital considerations exist when designing and implementing a reward function for a typical mobile robot [29], [30]. First, the reward function must clearly define the task's objective. The second is deciding whether a reward function should be rewarded only at specific milestones or continuous feedback, which is justifiable based on the concepts of sparse or dense rewards. The third is reward shaping, which involves adding auxiliary rewards to guide the robot's learning. The fourth is scaling and normalization. This tactic ensures that the scale of your rewards is appropriate, since if a reward function is not well balanced, it can affect the entire learning process of the robot system.

Another essential aspect worth considering when designing reward functions is the avoidance of reward hacking, and this entails being very cautious about developing a reward function that can be exploited by the robot with the intent or aim of achieving high rewards without actually solving the task. An agent acts to select actions to maximize its reward received from the environment [2]. Also, reward hacking can lead to undesirable behavior in the long run. Dense rewards are more susceptible to reward hacking [22], in which the reinforcement

learning agent utilizes its knowledge of the reward function to continue receiving rewards without completing the task, i.e., reaching the goal. Sparse rewards, however, may not provide enough information to guide the agent towards its goal efficiently.

Domain knowledge is another important consideration since incorporating domain knowledge, specifically from expert knowledge, when designing the reward function, can help craft meaningful reward signals. Lastly, benchmarking is another significant aspect of reward function design. It is vital to compare different reward functions and learning algorithms to find the most effective combination for the given robot's task. Finally, it is essential to note that designing an appropriate reward function can be a challenging and iterative process. The design process often requires a deep understanding of the problem domain and the learning algorithm's characteristics. The reward mechanism forms the basis for the RL agent's trial-and-error approach to learning policies [8]. From Figure 4 we can see that mobile robots are a subset of robots with its own unique state, action, and reward formulations.

2.5 Importance of reinforcement learning in mobile robot navigation

Reinforcement Learning (RL) provides a mathematical framework for developing, designing, and learning an agent's internal model through repeated trials [5], [31]. This trial-and-error learning allows for agents to continuously improve over time. Consider the case of learning to navigate from scratch; you are placed in an environment where you have no information. In reinforcement learning, the agent is dropped into an environment without any knowledge of how the dynamics work. In the absence of models for environment dynamics or priors input by the agent designer, mapless navigation has emerged to solve navigation problems with minimal assumptions regarding environment [32], [33].

In a rule-based navigation scenario, an agent's internal model is deterministic; that is, for a given observation x the output $f(x)$ is always the same. $f(x)$ is a function mapping the observations to actions; in a mobile robot, the observation x is obtained through sensory modules. An example of f can be seen below. In this example, x is a vector of range readings.

The reinforcement learning paradigm is a way to learn navigation policies through interaction.

$$f(x) = \begin{cases} Move & \text{if } x >= a \\ Stop & \text{if } x < b \end{cases} \tag{1}$$

The values of a and b are parameters that specify when the agent should perform each of the actions. In this case, the robot designer determines the values of a and b. The agents use this function to determine when to act; if the agent is close to an obstacle, it may execute the 'Stop' action, which stops the agent's actuators. The agent executes the 'Move' action when the range readings are above a certain threshold, meaning that there is free space to move without collision with obstacles. Figure 5 illustrates our simplified agent example.

Consider the agent depicted in Figure 6. One may collect samples of (x, y) and form a training set to learn the policy in a supervised fashion. x denotes a vector of features, and y denotes the correct action. Training an agent to perform the task requires a sufficient number of sample trajectories so that the agent can learn a robust policy. Reinforcement learning aims to abstract the interaction between an agent and its environment into objects that

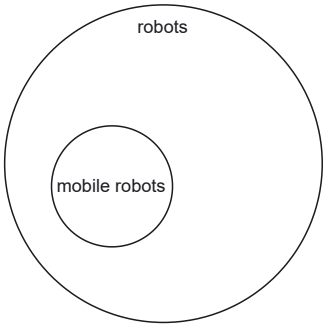

Figure 4: Mobile robots are a subset of robots that can move in their environment. Figure created by authors.

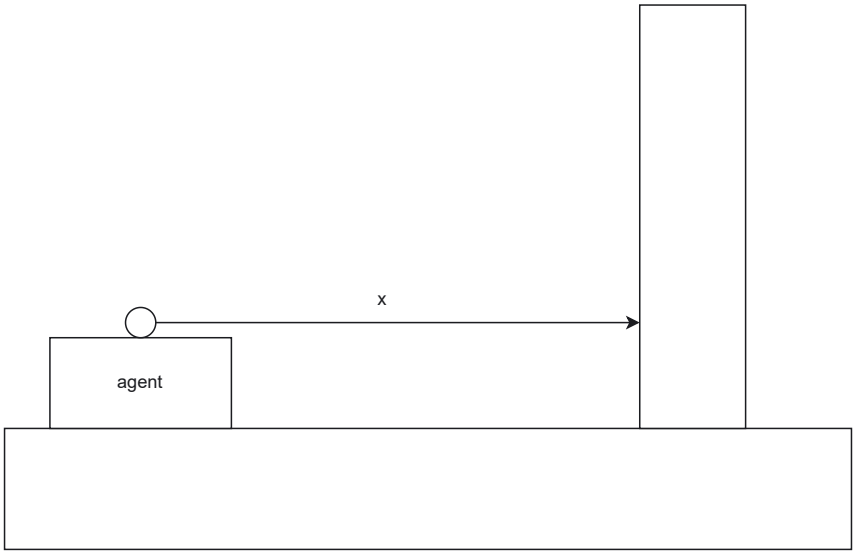

Figure 5: The agent observes x in its environment. In the example depicted $x \in \mathbb{R}$. Figure created by authors.

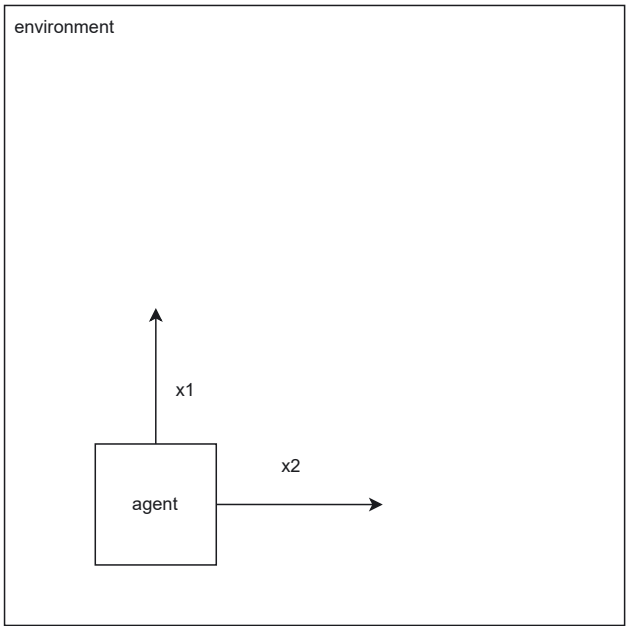

Figure 6: The agent observes x in its environment. In the example depicted $x \in \mathbb{R}^2$. Figure created by authors.

can be optimized. Unlike supervised learning, reinforcement learning agents learn through trial-and-error exploration and interaction with the environment the agent finds itself in; furthermore, in the supervised setting, the model's parameters are adjusted to minimize a loss function, while, in the RL setting, the agent is tasked with maximizing a reward function [31]. The agent is unaware of the reward function; it only receives a reward after acting in the environment.

We require mathematical abstractions for the state space, action space, and reward signal to optimize the parameters of a model-free RL agent. The state space for a mobile robot that navigates between two points consists of any information that may be vital for solving the task. The agent may use a camera's visual observations, allowing it to 'see' how its environment is structured. The agent has no understanding of what an obstacle is, what a table is, or what a wall is; it does not know its environment, and as such, the image may only serve to confuse the agent, i.e., images are high-dimensional input. A lower-dimensional input that works well for abstracting the environment.

3 Preliminaries

The field of reinforcement learning for mobile robot mapless navigation has seen significant advancements in recent years. In this review, we will discuss the latest developments and their impact on improving the navigation capabilities of mobile robots. We will explore various techniques to tackle this challenging problem, from deep reinforcement learning algorithms to hybrid approaches. We begin this section by presenting a comprehensive overview of the critical components and definitions of mobile robot navigation, followed by an introduction to reinforcement learning and a discussion of its key concepts and algorithms.

3.1 Definition and components of mobile robot navigation

Robot navigation is the ability of a robot to determine its position in its environment and then plan a path to the desired goal location [34]. This is a complex task, as robots must be able to sense their surroundings, understand the environment, and decide how to move. There are many different techniques for robot navigation, but they can be broadly divided into two categories: global navigation and local navigation. Global navigation refers to the general position of the robot in the environment [35], [36]. This is typically done by using a map of the environment, which can be created using a variety of sensors, such as GPS, laser scanners, or cameras. Once the robot has a map of the environment, it can use it to plan a path to its goal location. Local navigation is concerned with the robot's immediate surroundings. This is typically done using sensors to detect obstacles and other objects in the robot's path. The robot then uses this information to plan a safe path to its goal location. The following are some of the most common techniques used for robot navigation.

- **Path planning**: This is finding a safe and efficient path between the robot's current and goal locations.
- **Obstacle avoidance**: This detects and avoids obstacles in the robot's path.
- **Localization**: This determines the robot's position in its environment.
- **Mapping**: This creates a map of the robot's environment.

Robot navigation is a complex and challenging field, but it is also a rapidly growing field with many exciting applications. As robots become more sophisticated, the need for effective robot navigation techniques will only increase. Subsequently, we briefly discuss some well-known traditional mobile robot navigation techniques.

Inductive loop is a traditional robot navigation technique commonly used in indoor environments, such as warehouses, factories, and other controlled settings [37]. It involves using inductive loops, which are electromagnetic coils embedded in the floor or placed on the surface. These loops create a magnetic field that robots equipped with the appropriate sensors can detect and utilize for navigation. More so, the inductive loop navigation system operates on the basis of electromagnetic induction. An electric current that passes through the coils generates a magnetic field. When a robot with a receiver (inductive sensor) approaches the loop area, the magnetic field induces a current in the robot sensor.

The robot can then detect the presence of the loop and use it for navigation [38], [39]. They are typically installed beneath the floor's surface, making them unobtrusive and safe for humans and robots. The loops are laid out in specific patterns to create reference points and guide the robot through the environment. The navigation process using inductive loops generally involves calibration, detection, localization, path planning, and feedback control. Generally, this navigation approach provides the merit of high accuracy, reliability, and simplicity. However, their installation requires some upfront infrastructure costs, including embedding the loops in the floor, which can be a limitation for some applications. Furthermore, the inductive loops are greatly limited in terms of installation flexibility since once the loops are installed, the robot's navigation is confined to the areas covered by the loops. Adapting to environmental changes may require reconfiguring the loop layout [40].

Magnetic navigation is a robot navigation method that detects and interprets magnetic fields for positioning and orientation. This technique is beneficial in environments where traditional methods such as GPS or visual-based systems may not be effective or available [41], [42]. Magnetic navigation is often used in indoor settings or

areas with distinctive magnetic patterns that robots can use as reference points. Its principle of operation is based on the view that many indoor environments have unique and stable magnetic field patterns. These patterns can arise from various sources, such as metal structures, wiring, or natural variations in the Earth's magnetic field. Robots with the appropriate sensors can detect and use these magnetic fields for navigation.

A robot must be equipped with magnetic sensors capable of detecting and measuring magnetic fields to enable magnetic navigation. These sensors can include magnetometers, Hall effect sensors, or fluxgate magnetometers. The robot control system processes the sensor data to determine its position and orientation relative to the magnetic field patterns [43]. The navigation process in magnetic navigation involves calibration, detection, localization, path planning, and feedback control. They are equally cost-effective, as implementing magnetic navigation often requires minimal additional infrastructure, making it a cost-effective option for indoor robotic navigation [44]. Notably, this navigation technique can be greatly influenced by nearby metallic objects or other magnetic sources, leading to potential interference with the navigation process. Their effectiveness can be more limited to a confined area with distinct magnetic patterns. Moreover, calibration is crucial for accurate magnetic navigation, and environmental changes may require periodic recalibration.

Line following is a popular and widely used technique in robotics, where a robot autonomously follows a path or trajectory marked by a visible line on the ground [45]. This method is commonly employed in various applications, such as industrial automation, logistics, and educational robotics. Line-following robots utilize sensors to detect the line's position relative to the robot's current location and make appropriate control decisions to maintain the desired trajectory. The principle of line following is relatively straightforward. The robot has one or more sensors that can detect the contrast between the line and the background surface. These sensors can be infrared, reflectance, or colour, depending on the type of line used (e.g., black line on a white surface or vice versa).

By continuously monitoring the sensor data, the robot can determine whether it is directly over the line, to the left, or to the right [46], [47]. Their implementation can involve the use of the following commonly available components: line sensors, controllers, and actuators. Line following is relatively simple to implement and can be a good starting point for beginners in robotics. The components required for line following are often affordable and easily accessible. Also, line-following robots can be used in various applications, including automated material handling, warehouse logistics, and educational purposes [48]. However, this technique is highly limited because it relies heavily on the presence of a visible line. The robot may struggle to navigate if the line is damaged, interrupted, or not well-defined. Moreover, robots relying on this navigation method are restricted to the predefined path marked by the line, making them less adaptable to dynamic or unstructured environments.

Vision-based navigation, or visual navigation, is a robotic technique that uses computer vision to enable robots to navigate and interact with their environments using visual information [49], [50]. This approach involves capturing images or video streams from cameras mounted on the robot and processing these visuals to extract relevant data about the surroundings. Vision-based navigation has gained significant importance in robotics due to advancements in computer vision algorithms, camera technology, and machine learning techniques. The principle of vision-based navigation revolves around analyzing visual data to extract meaningful information about the environment. The robot's cameras capture images or video frames, which are then processed through computer vision algorithms to recognize objects, features, and landmarks.

This information is used to make navigation decisions, avoid obstacles, map the environment, and localize the robot's position [51]. Implementing vision-based navigation typically involves using the following components: high-quality cameras, computer vision algorithms, mapping and localization, path planning, and feedback control. This flexible technique provides information about the environment and enables robots to make more informed navigation decisions. They can also provide real-time perception of dynamic environments, making them suitable for scenarios with rapidly changing conditions [52], [53]. Some of its limitations include sensitivity to lighting conditions, high computational complexity, and occlusion.

Ultrasonic navigation is a robotic technique that uses ultrasonic waves to enable robots to navigate, avoid obstacles, and determine distances to objects in their surroundings. Ultrasonic navigation is commonly used in various applications, including mobile robotics, autonomous vehicles, and robotic sensors [54]. The method utilizes ultrasonic sensors to send and receive sound waves, analyzing the reflected signals to make navigation

decisions. Generally, ultrasonic navigation is based on the principle of echolocation, similar to how bats navigate in the dark. The robot emits ultrasonic waves, which are sound waves with frequencies higher than the human hearing range (typically above 20 kHz). These waves travel through the air, and when they encounter an obstacle or object, they bounce back as echoes. The robot's ultrasonic sensors then detect and measure the time it takes for the echo to return.

By analyzing this time delay, the robot can estimate the distance to the obstacle and determine its position relative to it [55], [56]. The method is typically implemented using the following components: ultrasonic sensors, controllers, and actuators. Some of the method's advantages include being relatively simple and inexpensive, making them accessible for various robotic applications. The method provides real-time data about the robot's immediate surroundings, allowing for responsive and dynamic navigation [57]. However, this method has a limited range, typically a few meters. Beyond this range, accurate distance measurements become challenging. They can also be affected by reflections and surface interference, leading to inaccuracies in distance measurements. Lastly, they may face challenges in certain environments with complex acoustic properties or areas with a lot of background noise.

In this work, we are primarily concerned with how robots navigate an environment given sensor data. The remainder of this paper is dedicated to how navigation is achieved through the use of RL. Similarly to how sentient beings learn to navigate by trying various options until one is found that solves the problem, the RL agent is placed in an environment and must learn through trial and error what the rules are that govern the environment in which it finds itself. When tasked with navigation from a start state to a goal state, we enter the realm of goal-driven navigation. When the agent is equipped with mechanisms for building and updating a map of its environment, as in the Simultaneous Localization and Mapping (SLAM) algorithm, we enter the domain of map-based navigation approaches.

The SLAM algorithm has emerged as a popular way for solving robot motion planning and control and consists of estimating an agent's trajectory and a map of the environment as it moves [58]. The generation and maintenance of a map means that the agent is essentially hard-coded or overfitted to the environment for which it was designed. Furthermore, the generation and maintenance of an online map may be infeasible in resource-constrained environments. To illustrate, consider an agent trained to navigate environment A, when placed in environment B, it would need to start learning from scratch as opposed to adding to its existing knowledge base. To increase data efficiency, modern RL approaches use a replay buffer of past experiences to update the parameters of the policy. This implies that if the agent were dropped in an environment it was not trained on, if the policy was able to learn well enough during training, the agent could perform even if the environment was not seen during training. The use of RL allows for the learning of generalizable and data-efficient navigation policies through trial and error.

3.2 Introduction to reinforcement learning

Reinforcement learning is a learning paradigm in which an intelligent agent placed in an environment can learn to achieve its goals through continuous interaction. Reinforcement learning is distinguished by its trial-and-error approach, which involves selecting from various actions at each step and learning to associate observed states with the most appropriate actions [59]. In the context of a reinforcement learning environment, an intelligent agent seeks to find an optimal policy to solve a sequential decision-making task [60]. A policy, denoted as π is a function that outputs the correct action to perform given the current state. A stochastic policy denoted $\pi(s, a)$ is the probability of selecting action a in state s. An agent placed in an environment is defined by the tuple $\langle \mathrm{S}, \mathrm{A}, \mathrm{P}, \mathrm{R}, \gamma \rangle$.

- S denotes a set of states that the agent observes through sensory input on the board.
- A denotes a set of actions available to the agent in the environment.
- P denotes a matrix of transition probabilities. Each entry P_{ij} is the probability of moving from state i to state j.
- R is the reward function; a scalar function $R(s, a)$, which quantifies the desirability of performing action a in state s.

At each timestep, the agent observes its state s_t, selects an action given this state a_t, performs the action, and observes a scalar reward r_{t+1} and state s_{t+1}. The agent's goal is to maximize the reward it receives in a single trajectory. If the reward signal is maximized, it is assumed that the agent is performing the task in the desired way; as can be seen, this may not always be the case. Consider the cart pole environment; the task is to balance a pole vertically on a cart that moves to the left and right. The pole is kept vertical by applying a push in the opposite direction from which the pole leans. In this task, the agent receives a reward unit for every second it keeps the pole upright. In the cartpole environment, the agent receives a reward at every timestep, so the reward signal is dense. Dense reward signals provide information at every time step, unlike sparse reward signals, which only reward the agent at the end of the episode. Hand-designed reward functions are difficult to design properly, making it even more challenging as the environment becomes more complex. Hand-designed reward functions require knowledge of the dynamics of the task being solved. Figure 7 illustrates how an agent is connected to its environment.

3.3 Key concepts and algorithms in reinforcement learning

This section introduces the reader to key concepts and algorithms used to solve reinforcement learning tasks. A reinforcement learning agent is linked with its environment through states and actions. It is not possible to define one without the other. The environment in which the agent finds itself determines which states it will observe and which actions are possible. Therefore, a careful consideration of both the agent and the environment in which it is placed is needed to achieve success in the given task.

3.3.1 Markov Decision Processes (MDPs)

A Markov Decision Process (MDP) is a mathematical framework for describing and solving sequential decision problems [61]. An MDP is defined by a state and action space that interact to learn the underlying environment dynamics to maximize a numeric signal. The MDP framework decouples goal-directed tasks into three signals that pass back and forth, i.e., the action performed a_t, the basis on which the action was selected s_t, and the goal r_t [62]. The agents' goal is to maximize reward. The decisions an agent takes at timestep t influence the following states the agent may encounter and which actions are available at timestep $t + 1$.

$$s_{t+1} \sim P(s_{t+1} | (s_0, a_0), (s_1, a_1), \ldots, (s_t, a_t)) \tag{2}$$

The Markov property states that deciding which action to take next only needs to consider the previous state and action, and not the entire history of states and actions.

$$s_{t+1} \sim P(s_{t+1} | s_t, a_t) \tag{3}$$

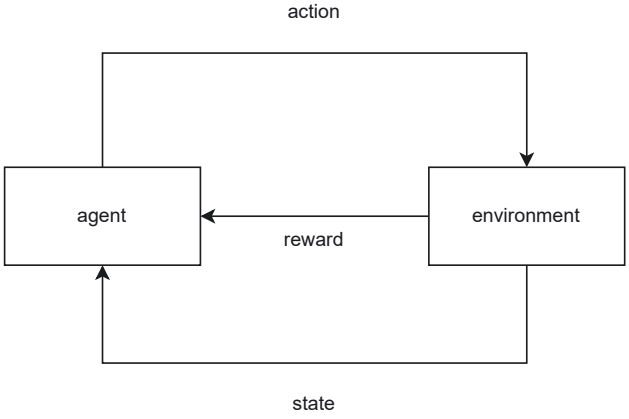

Figure 7: Agent-environment interaction. Figure created by authors.

Consider a game of pool in which the objective is to use the white ball to shoot the rest of the balls into one of the six holes. In this example, the layout of the balls on the table at timestep $t+1$ depends on the balls' configuration at t. In timestep t, the set of available actions is denoted as A_t. Knowledge of how the balls were configured on the table at $t-1$ does not provide information on which action to select at $t+1$. This interrelation of states and actions is quantified using the Bellman equation. In Equation (3), the probability of transitioning to state s_{t+1} given that you were in state s_t and took action a_t, this probability is rarely known in practice. The probabilities become infeasible to estimate as the task to be solved becomes more complex, i.e., an increase in the number of actions in A or states in S.

To select the best action to take at time t one must define what makes one state s_a better or worse than another state s_b. The value estimate estimates how much reward an agent can receive starting in a state s and following a fixed policy π; the policy π is a function that maps states to actions. To estimate your next state s_{t+1}, you only need information about s_t, following the Markov assumption.

The value of a state is an agent's estimate of how 'good' it is to be in a particular state [62].

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[G_t|s_t = s] \\ &= \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}|s_t = s\right] \end{aligned} \tag{4}$$

Equation (4) is the definition of value; the expected value of starting in a state s and following policy π until the end of the episode. The value of starting in a particular state is the expected discounted return, i.e., how much reward can one expect to receive from this point onward? Given optimal action selections, states with low values will be less likely to be chosen. The action value estimates the expected value of starting in a state s, performing an action a, and following π till termination [62].

$$\begin{aligned} q_\pi(s,a) &= \mathbb{E}[G_t|s_t = s, a_t = a] \\ &= \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1}|s_t = s, a_t = a\right] \end{aligned} \tag{5}$$

Note that the action value has almost the exact definition of value, except that the distribution is conditioned on which action the agent selects at the start.

$$a_t \sim \pi(s_t) \tag{6}$$

The action value estimate Q estimates the value of being in a state s_t and acting a_t. The action value estimate $Q(s_t, a_t)$ contains information about state-action pairs, whereas the value estimate $V(s_t)$ only takes into account the state s_t [61]. Reinforcement algorithms are centered on learning a $Q(.)$ or $V(.)$ function. To estimate the value of a state s we require knowledge of how the environment evolves. Learning a Q function is preferred to learning a V function because it allows us to estimate the future value using experience. Furthermore, the Q function is defined over states and actions, allowing one to represent the value of each action.

3.3.2 Value-based methods

Value-based RL algorithms estimate the expected return from trajectories. A value-based algorithm learns a value function and selects its actions by maximizing the value.

$$V_\pi(s) = \mathbb{E}[G_t|s_t = s] \tag{7}$$

This section consists of two methods for *prediction* and *control*. Prediction refers to estimating the value of states, and prediction refers to using the value predicted to select actions. Both SARSA and Q-learning are examples of Temporal-Difference (TD) learning. Temporal difference methods make the update $V(S_t) = V(S_t) + \alpha(R_t + \gamma V(S_{t+1}) - V(S_t))$; α is the learning rate or step size, R_t is the reward obtained in timestep t, this is the reward for transitioning from state s_t to s_{t+1}. The value of the current state is updated using the difference between the current state and the expected value of the next state. The return G_t is estimated using $R_t + \gamma V(S_{t+1})$.

policy			
	state A	state B	state C
action 1	Q(A, 1)	Q(B, 1)	Q(C, 1)
action 2	Q(A, 2)	Q(B, 2)	Q(C, 2)
action 3	Q(A, 3)	Q(B, 3)	Q(C, 3)

Figure 8: Tabular Q-Learning stores the policy as a lookup table. Figure created by authors.

$$G_t = \sum_{k=0}^{\infty} \gamma R_{t+k+1} \tag{8}$$

The benefit of using G_t estimates is that one can estimate the return at each step instead of waiting till the end of the episode to update the value of $V(S_t)$. The key difference between *Monte − Carlo* RL methods and *TD* methods is how the value of a state is estimated; this is known as the *prediction* problem [62]. Temporal difference methods have the advantage of learning, i.e., updating estimates at every timestep. Monte-Carlo methods wait until the end of an episode before updating estimates since G_t is only available at the end. Figure 8 illustrates how Q-value estimates for a simplified agent can be stored in a table.

Q-Learning updates each Q-value estimate using Equation (9). The max in Equation (9) means that Q-learning implicitly acts greedily by selecting an action a_t in state s_t and then using the maximum action value to update the value estimate of s_t. Q-learning converges to the optimal policy q_* by the max operator.

$$Q(s,a) = Q(s,a) + \alpha\left[R + \gamma \max_a Q(s',a) - Q(s,a)\right] \tag{9}$$

SARSA updates each state action value using Equation (10).

$$Q(s,a) = Q(s,a) + \alpha[R + \gamma Q(s',a') - Q(s,a)] \tag{10}$$

The name SARSA is derived from the *experience* tuple that is obtained at each timestep, namely $\{s_t, a_t, r_t, s_{t+1}, a_{t+1}\}$ [62]. The term *on-policy* means SARSA learns on the job.

Both Q-learning and SARSA update their respective estimates at every timestep. The key difference is that SARSA selects a_t and a_{t+1} using a policy derived from Q. Q-learning selects an action a_t using a policy derived from Q, but the second action is selected using a maximum over all the possible actions from S_{t+1}.

Deep Q-Network (DQN) is the combination of deep learning and reinforcement learning principles to create efficient algorithms [63]. Deep Q-Networks (DQN) parameterize a policy using a set of weights θ [3] to efficiently handle large state and action spaces. At each timestep, the agent stores its current experience in a memory buffer. The buffer is sampled, and the batch is used to update the policy parameters to minimize the loss defined in Equation (11).

$$L(\theta_t) = (r_t + \gamma \max_a \overline{Q}(s_{t+1}, a_t, \theta_t) - \overline{Q}(s_t, a_t, \theta_t))^2 \tag{11}$$

Learning the Q-function with a single neural network causes instability during learning. Double Deep Q-Networks (DDQN) is proposed to stabilise the learning of the value function. The DDQN algorithm uses two networks; one network is used to estimate the expected discounted future reward, and the other is used to estimate the state-action value of the current state. Learning stability is further enhanced using a memory buffer that stores transitions in memory to sample from in batches. The batches of transitions are used to update the neural network parameters by backpropagating the error $L(\pi_\theta)$ w.r.t., the parameters of the network θ. Furthermore, sampling a random batch of transitions removes the temporal correlation between the target values $r_t + \gamma \max_a Q(s_{t+1}, a)$ and the current action value estimates Q [3].

Using a neural network allows one to process states in textual format, images, vectors, matrices, and a combination. Neural networks are universal function approximators. Using a neural network to parameterize a policy allows for Reinforcement Learning (RL) methods to be applied to a variety of environments, i.e., we are no longer limited to tabular environments. Furthermore, using a neural network allows for processing multimodel state representations.

The core idea behind the DQN is to shift the action values Q predicted by the neural network towards their expected value predicted by the Bellman equation $r_j + \gamma \max_a \widehat{Q}\left(\phi_{j+1}, a\,; \theta^{-}\right)$. The parameters θ^{-} are kept fixed to avoid the 'moving target' problem in which the estimates are moved towards a value that itself keeps moving. For every C step, the parameters are copied.

MaxPain

In the MaxPain [60] setting, the reward R is decomposed into two separate streams: one for positive rewards and one for *punishment*. A single reward stream influences the agent's behaviour in the traditional Q-learning setting. Splitting the policy into two streams is inspired by results in neuroscience, showing that biological systems have separate mechanisms for processing pain and pleasure [60]. Formally *reward* is defined as $r = \max(R, 0)$ and pleasure $p = -\min(R, 0)$. The max in the definition implies that the reward is associated with a positive experience by the agent, which is in line with most reward formulations, i.e., the agent gets a positive reward for good behaviour and a negative reward for bad behaviour.

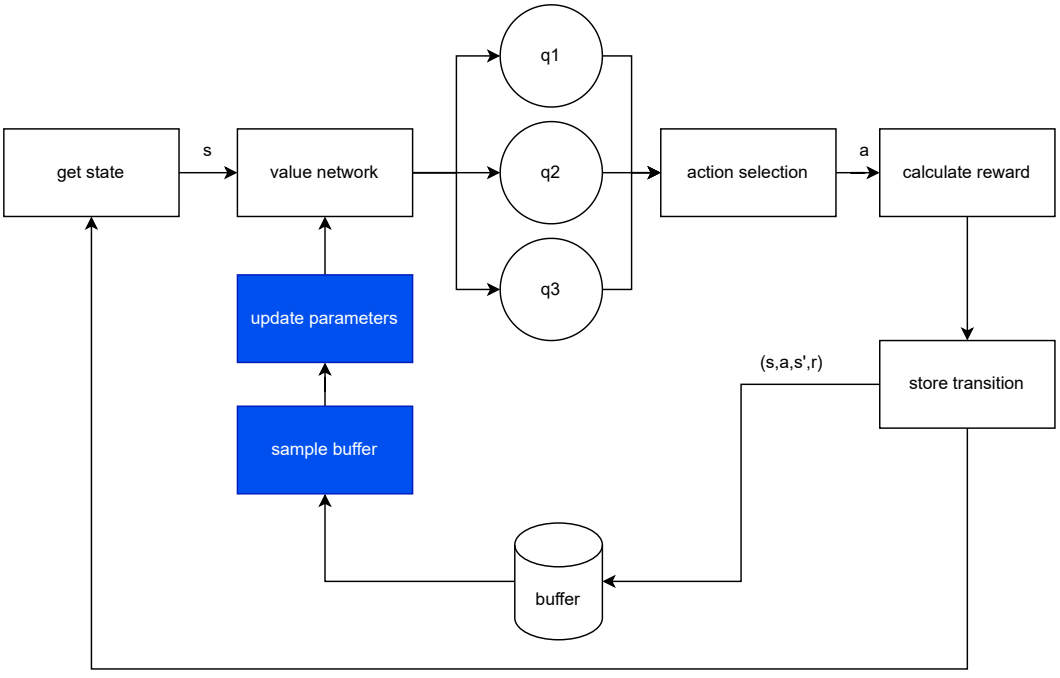

Figure 9: Flow of information in a value-based DRL system. Figure created by authors.

The MaxPain algorithm learns and maintains estimates of two separate action value functions and combines the outcome of the two using a weight $0 \leq w \leq 1$. The term w controls the trade-off between maximizing reward and minimizing punishment.

$$Q_w(s,a) = wQ_r(s,a) - (1-w)Q_p(s,a) \tag{12}$$

In the MaxPain algorithm, the TD-error can be the off-policy Q-learning or on-policy SARSA from Equation (9) and (10), respectively. Figure 9 illustrates the flow of information in a value-based DRL agent.

3.3.3 Policy based methods

Policy-based methods learn a policy directly from interaction instead of learning an intermediate function, i.e., learning the value function and selecting actions based on the value. The policy is stochastic versus the deterministic policy learned from value-based methods. Policy-based methods for reinforcement learning in mobile robot mapless navigation aim to learn a policy, or a set of rules, that dictate the actions a robot should take in a given environment. This approach involves directly optimizing the policy rather than trying to learn the value function, which represents the expected long-term return for a given state-action pair.

Some common policy-based methods used in mobile robot mapless navigation include policy gradient methods, evolutionary algorithms, and Bayesian optimization. Furthermore, [23] best summarises this approach as follows: the policy-based methods use function approximation to create a policy network, which then selects actions to maximize the reward value. The parameters of the policy network are optimized in the direction of the gradient to obtain an optimal policy that maximizes the reward value [64].

A popular policy-based mobile robot mapless navigation method uses deep reinforcement learning algorithms [65]. These algorithms combine the power of deep learning with the reinforcement learning framework, allowing for more complex and accurate policies to be learned. Deep reinforcement learning has been successful in tasks such as obstacle avoidance and path planning for mobile robots and is constantly being improved and refined. Another approach to policy-based methods for mobile robot mapless navigation is using hybrid methods. These methods combine reinforcement learning with other techniques, such as imitation learning or pre-trained policies, to improve learning efficiency and performance. For example, a hybrid approach may use a pre-trained policy for basic navigation and reinforcement learning to fine-tune the policy for specific environments. This approach has shown promising results in various mobile robot navigation tasks and is an area of ongoing research [66], [67].

Policy gradient methods are the most popular class of RL algorithm for continuous action spaces, this class of algorithm uses gradient ascent to modify the policy parameters directly [68]. A policy gradient agent updates the parameters of its policy π using Equation (13).

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}\left[Q^\pi(s,a)\nabla_\theta \log \pi_\theta(a,s)\right] \tag{13}$$

The value $\pi_\theta(a, s)$ on the LHS denotes a *stochastic* policy and can be approximated by a model; in deep reinforcement learning, neural networks are used. In the case of a stochastic policy the network outputs a distribution over actions, during action selection this distribution over actions is sampled.

3.3.4 Actor-critic methods

Actor-critic methods are a popular approach for reinforcement learning in mobile robot mapless navigation [69], [70]. This method combines the advantages of policy- and value-based methods, using a separate network for each. The ”actor” network learns the policy, while the ”critic” network learns the value function. These two networks work together to optimize the policy and improve learning efficiency. One of the key advantages of Actor-Critic methods in mobile robot mapless navigation is their ability to handle continuous action spaces. This is particularly useful for mobile robots, as they often operate in environments with many possible actions. In addition, Actor-critic methods are more stable and efficient than other approaches, making them a popular choice for reinforcement learning in mobile robot navigation. Moreover, [71] proposed and used an actor-critic RL

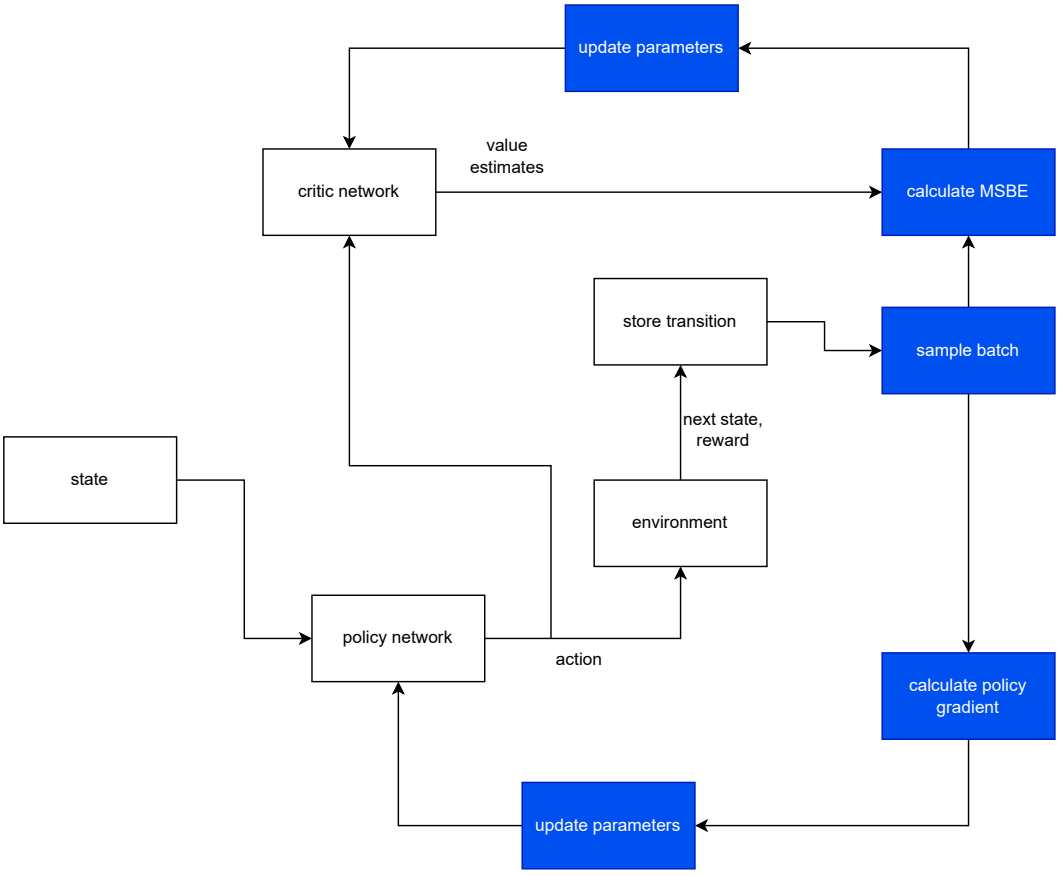

Figure 10: Flow of information in an actor-critic DRL system. Figure created by authors.

framework for control, which can guarantee closed-loop stability by employing the classic Lyapunov's method in control theory. However, they do require careful tuning of parameters and network architectures for optimal performance. Figure 10 illustrates the high-level flow of information in an actor-critic DRL agent.

Deep Deterministic Policy Gradient (DDPG)

Given the success of DQN in combining Deep Learning with Q-learning to tackle problems with high-dimensional state and action spaces, Lillicrap et al. [72] proposed the Deep Deterministic Policy Gradient (DDPG) algorithm DDPG uses a deterministic actor $\mu(s|\theta)$ to propose actions. The *critic* is used to estimate the value Q of the action output from the *actor*; algorithms of this configuration are known as actor-critic methods. DDPG learns a policy π parameterized by a set of parameters θ, to stabilize training of π DDPG uses a target network $\theta^{\mu'}$. The target network parameters are updated every k timesteps by copying the parameters of the policy network θ^{μ}. Silver et al. [68] show that for a deterministic policy the gradient of the agents performance measure can be estimated using Equation (14), calculated using a batch of transitions sampled from the agent's replay buffer. In Equation (14) the values $\mu(s|\theta^{\mu})$ and $Q(s, a|\theta^{Q})$ are represented using actor and critic networks with parameter vectors θ^{μ} and θ^{Q} respectively.

$$\nabla_{\theta^{\mu}} J \approx \frac{1}{N} \sum_{i} \nabla_{a} Q(s, a|\theta^{Q})|_{s=s_i, a=\mu(a))} \nabla_{\theta^{\mu}} \mu(s|\theta^{\mu})|_{s=s_i} \tag{14}$$

Twin Delayed Deep Deterministic Policy Gradient (TD3)

To address the limitations associated with value estimation – namely over-estimation and accumulation of errors, Fujimoto et al. [73] introduce the Twin Delayed DDPG (TD3) algorithm. The Q – function suffers from over-estimation as a result of selecting actions greedily with respect to the highest expected return, known as the overestimation bias. In [73], the authors show that the overestimation negatively influences the performance of actor-critic algorithms. To address the limitations of previous continuous action space methods, TD3 learns two q-functions and selects the minimum of the two to update their parameters. This mechanism of using the lowest Q-value has the added benefit of ensuring that value estimates are more conservative, reducing the effects of overestimation. In their work [73] note that the agent benefits from less frequent updates to the policy, the authors use 1 update to the actor for every 2 updates to the critic. Furthermore, TD3 introduces 'target-policy smoothing' in which noise is added to the output from the target network. In their work the authors of TD3 note that when updating critic parameters using a deterministic target, the updates are negatively influenced by function approximation error. Traditionally action noise has been added to actions output directly from the policy, in the TD3 algorithm noise is added to the target that the agent shifts value estimates towards. Equation (15) defines the critic's target, smoothed by adding noise ϵ. The value $Q_{\theta'}$ represents the target Q-Network with parameters θ.

$$y = r + \gamma Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon) \tag{15}$$

Soft Actor-Critic (SAC)

In addition, the Soft Actor-Critic (SAC) is a variant of the Actor-Critic method that has been proposed, and its core idea is to use approximation functions that can learn continuous action space policies, characterizing this method as a stochastic actor-critic [74]. The SAC algorithm modifies the traditional RL objective with an entropy term to encourage exploration of its agent's state space. The soft actor-critic uses a maximum entropy objective in the value and policy to prevent premature convergence of the value function and to ensure exploration by rewarding actions that lead to states with high entropy [75]. Equation (16) defines the entropy regularized objective used in SAC.

$$J(\pi_\theta) = \sum_{t=0}^{T} \mathbb{E}_{(s_t, a_t)\sim\pi_\theta}[r(s_t, a_t) + \alpha\mathcal{H}(s_t)] \tag{16}$$

Value-based methods learn a function to estimate the value of a given state, which is then used to select actions [31]. With value-based methods, an agent indirectly learns to select the best actions to achieve its goal. This indirect learning is different from the policy-based methods that directly modify the parameters of the policy to achieve its goal [2]. This direct strategy is different from the indirect one as the same policy used to select actions is the one that gets updated. Actor-critic methods combine the best parts of value-based and policy-based methods by using an approximator that selects actions called the **actor**, and another approximator measures the suitability of the actions taken, called the **critic**. In this work, we aim to understand how each of these methods has been applied to achieve the navigation of an agent in an environment.

The key difference in the approaches mentioned is how the solution is represented; value-based methods represent the policy as a value function. Value-based methods do not contain implicit exploration in their design and require a separate policy for action selection, e.g., selecting the action with the maximum value or epsilon-greedy action-selection. Policy-based methods represent the solution as a policy and can often achieve a better exploration of an environment as a result of directly modifying the policy parameters. Q-learning emerged as a tabular method for temporal-difference learning. DQN went a step further by now representing the policy as a function approximator and using a replay buffer to update the parameters of the policy; the original DQN was limited to discrete action spaces. DDPG emerged as a method for learning policies in continuous action spaces. The SAC algorithm goes a step further by adding entropy into the objective being maximized, which allows for efficient exploration as the agent is always selecting actions that maximize the expected reward and entropy. The SAC algorithm is an example of maximum entropy RL.

In policy-based algorithms, the agent selects an action and evaluates its value under the current policy. The network parameters are then updated via the gradient ascent on the policy gradient. The DDPG algorithm

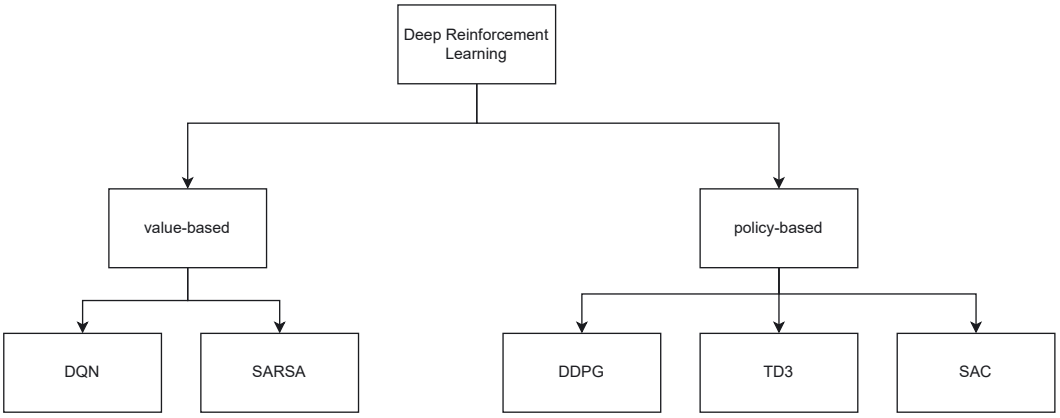

Figure 11: RL algorithm taxonomy. Figure created by authors.

employs a deterministic policy, producing the same action for a given state. In contrast, the SAC algorithm uses a stochastic policy, sampling actions from a probability distribution to explicitly influence action selection. The choice between these algorithm classes depends mainly on the action space available to the agent, since the reward function and state definitions can remain consistent across methods. A review of recent state-of-the-art problem formulations appears in Section 5. Figure 11 presents a taxonomy of RL algorithms.

4 Research methodology

This section highlights how we systematically reviewed the most recent work on mapless navigation to gain insight into the latest research trends. We establish search strings, search reputable literature sources, and used inclusion and exclusion criteria to extract suitable literature to provide a review of state-of-the-art reinforcement learning and its applications to mapless navigation tasks.

4.1 Keywords

The literature on reinforcement learning is vast, as it applies to various tasks. When searching for literature on mapless navigation and the most recent approaches to target-driven mapless navigation, specific keywords were employed to narrow the search and zoom in on only the most relevant literature. To obtain the relevant studies in the field of mapless navigation, we use keywords such as 'mapless navigation' and 'mapless navigation using reinforcement learning'. We used ' reward shaping ' to obtain papers related to some of the problem areas/recent fields of interest related to mapless navigation. Reward shaping is a technique used to specify an agent's reward, so naturally, all literature that appeared using 'reward shaping' was a subset of reinforcement learning. Mapless navigation of a mobile agent refers to navigation in an environment without a predefined map. To extend our review, we included the search term 'target-driven'. Searching for 'target-driven mapless navigation' refers to navigation in an unknown environment to reach a goal state.

4.2 Literature databases

To gather relevant literature for our research, we searched popular academic databases such as IEEE Xplore, ScienceDirect, PubMed, Scopus, Public Library of Science, EBSCOhost, Directory of Open Access Journals, JSTOR, and ScienceOpen. In addition, we utilized *connected papers* to visualize the connections between different articles. This tool allows for sorting search results by year, number of citations, and similar papers. We also

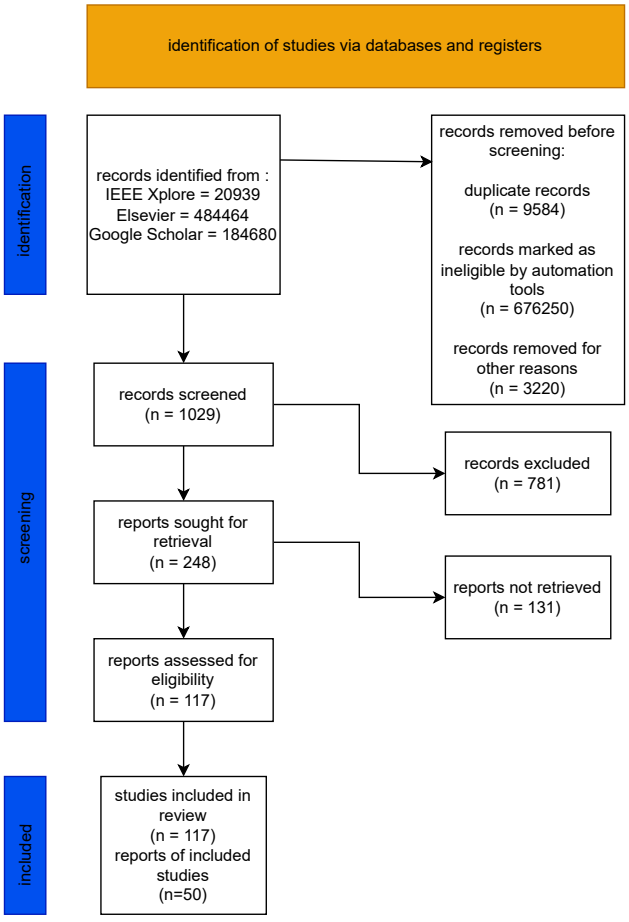

Figure 12: Reviewed article's search and selection process using PRISMA. Figure created by authors.

accessed open-source electronic libraries such as Google Scholar and MDPI, which provide a comprehensive range of literature covering various fields of science. Gathering literature from reputable databases is an essential first step in any inquiry, and we aim to explore the latest trends in mapless navigation to gain a deeper understanding of the field's recent developments.

4.3 Inclusion and exclusion criteria

Our focus in this work is to examine trends in mapless navigation systematically. Systematic reviews are considered the reference standard for evidence synthesis due to the rigorous methodologies they employ [76]. We included results that used traditional machine learning methods to gain a holistic view of all approaches. This provided a basis for exploring RL approaches, as we could now identify the weaknesses. We included results/papers using SLAM and Bayesian filtering techniques; these methods exhibit the potential to localize an agent while navigating an unknown environment. We found that the popularity of such approaches has recently improved, and having a view of such approaches would prove useful to the reader of this work. Our search of the literature focuses on recent articles and papers from 2019 to 2025, providing an overview of how the field has evolved in this period. Figure 12 presents the selection process of relevant articles using PRISMA (Moher et al., 2010) guidelines for reporting.

4.4 Comparison with previous reviews

Mapless navigation of robots using RL is an active field of research, and as such, a vast body of existing work exists. We use this section to discuss how our review relates to what exists in the literature.

In their review, Cebollada et al. [63] discuss the importance of visual sensory data and how it has been used for tasks like localization, feature extraction, and mapping specifically for robot navigation. A robot is connected to its environment through the sensors attached at the time of design; more so, in their review, Cebollada et al. [63] focused only on visual information and did not document other types of sensory data. Unlike this approach, we investigate how LiDAR data have been used in navigation. We also investigated methods that combine image and LiDAR for navigation.

Zhu et al. [23] review the RL algorithms and discuss challenges and techniques to overcome them. Similar to [23], we summarize recent DRL approaches in input type, action space, and reward setting. We add a dimension of analysis by investigating which variables have been incorporated into the reward to *shape* it. Furthermore, regarding action space design, we highlight exactly which values are output by the action selection policy.

Möller et al. [77] survey visual navigation techniques and strategies for human-aware robot navigation. In their work, the authors note that two kinds of computer vision exist in robotics, namely *active* and *passive* vision.

Chang et al. [78] focus on reviewing the techniques used in the autonomous navigation of Unmanned Aerial Vehicles (UAVs). The authors focus on map-based and mapless navigation and the various components that form a navigation agent, namely perception, planning, and control. In [78], the authors discuss various ways to combine image and distance data by using various submodules to navigate a drone in indoor environments where GPS localization may not be feasible. The paper focuses on CNN approaches; the authors do not document reinforcement learning approaches as extensively. AlMahamid et al. [79] review drone navigation from the reinforcement learning perspective, focusing on state-of-the-art algorithms. The authors classify RL algorithms based on the number of available states the algorithm can process and the type of actions available to the agent, and list a few areas of application. AlMahamid et al. [79] review RL algorithms and their application to UAV navigation.

We aim to systematically review the literature on the mapless navigation of mobile robots in unknown environments. As such, we start by reviewing reinforcement learning and how the navigation task has been framed as a Markov Decision Process. We identify pain points experienced by recent works, some of which have been identified by the authors, and others noted for future work. We focus on mapless navigation using reinforcement learning algorithms and review the latest approaches with a focus on how the state was processed, which algorithms were used, which reward formulations are common, and whether or not real-world transfer was done. A summary of recent surveys on different navigation methods can be found in Table 1.

5 Mapless navigation approaches

This section summarizes recent approaches to mapless navigation. We have separated this section into a discussion of navigation using LiDAR, image, and a combination of LiDAR and image data. The tables in this section aim to contrast the various ways in which the authors have formulated the task of mapless navigation. This section starts by highlighting the approach taken. It concludes with a discussion of the strengths and weaknesses of each approach, the aim being to identify trends and algorithms that have stood the test of time and to identify areas of improvement. At the end of this section of the paper, the reader will have insight into some of the problems RL algorithms face and how previous works have set out to combat some of these shortcomings. Furthermore, the reader will understand where mapless navigation is headed and which areas have shown promise.

Light Detection and Ranging (LiDAR) provides a cost-effective way to obtain information regarding the distribution of free space around an agent. LiDAR uses light produced by a laser to measure distances [80]. It should be noted that in the works [12], [81], [82] and more recently in [19], [83], [84], the LiDAR data is combined with goal and robot state information to make the state available to the agent for decision-making more

Table 1: Summary of recent surveys on navigation methods.

Year	Ref	Focus	Summary	Recommendation
2021	[63]	How can image data be used for feature extraction, localization, and mapping?	The authors summarize approaches that use image data for navigation in terms of task, input image type, and algorithm used for decision making	Cloud-computing and Big Data can efficiently train models from datasets. Increasing agents' autonomy will increase their adoption in other application domains.
2021	[23]	What RL algorithms are used for navigation and what problems and techniques are used to overcome them?	The authors summarize various DRL approaches to navigation in terms of input type, action space, reward setting, and application scenario.	Problems encountered during navigation; partial observability, poor generalization, and sparse reward are discussed, and possible solutions discussed.
2021	[77]	How is vision used in human-aware robot navigation?	The authors summarize recent DRL approaches to navigation of humanoid robots among humans in terms of problem description, algorithm used, and performance metrics used to measure success. Vision forms the basis for most of the research surveyed. The authors note that robot navigation datasets can be augmented to include humans	Motivate the need for socially aware robots. Recent works on navigation make use of datasets that can be modified to include humans via image augmentation. Robots need to be aware of the humans present in the environment in which they operate.
2022	[79]	How can an RL approach be formulated?	The authors summarize RL algorithms and their application to UAV navigation. Benchmarking, environment complexity, knowledge transfer, UAV complexity, and algorithm diversity are highlighted as problems and solutions discussed	The proposal of a six-step process for formulating an RL task, namely state definition, action definition, policy selection, processing type, number of agents, and algorithm selection.
2023	[78]	What techniques and tools are used for the navigation of UAVs	Mapless and Map-based navigation of UAVs in indoor environments using image data. The authors classify mapless navigation using images into distinct classes based on how the policies are derived. The authors compare several image-based approaches in simulation,	Three performance measures are proposed for evaluating navigation policies, namely, Path Length, Deviation Rate, and Exploration Efficiency. Future research should focus on developing open-source navigation datasets with reliable labels, algorithms that can learn by themselves, and high-fidelity simulators that can closely model reality.

informative. In their work [85] utilize traditional control theory to enhance their agents understanding of the environment in which it is placed. In the works of [86]–[88] and more recently [89] use the goal information directly in the state representation of their agent. Information regarding the goal is vital to navigate using LiDAR, as the range measurements do not provide any information regarding the goal.

Reward shaping and design allow for encoding certain behaviours into an agent. LiDAR data measures obstacle distribution around an agent; shorter LiDAR measurements correspond to obstacles close to the light source. Studies [90], [91] and more recently [83], [92], [93] utilize sparse reward mechanisms in which the agent only receives reward upon reaching its goal or collision with obstacles. Sparse reward mechanisms are straightforward to design and implement at the cost of being less informative to the agent. In the sample of articles extracted for review, most focus has been on reward shaping to provide environmental insight beyond the sparse distance readings.

Various approaches have been taken to formulate reward functions for achieving specific objectives, in the works of [86], [94], [95], the reward is augmented to achieve obstacle avoidance and goal reaching behaviour. To achieve more fine-grained control of agent behaviour [12] uses a pre-trained matching network, in [96], entropy and information gain are used to encourage exploration. The distance difference measures the distance to the goal over two consecutive timesteps. Scaled distance difference is the most popular reward mechanism for encouraging goal-reaching. When goal information is unavailable, navigation methods that rely solely on LiDAR

Table 2: Summary of recent approaches utilizing LiDAR.

Year	Ref	Algorithm	State	Action	Reward	Novelty
2019	[88]	DDPG	180-D LiDAR	**Continuous**: linear and angular velocity	**Dense**: scaled distance-difference	The inclusion of 1-D convolution to encode LiDAR
2020	[14]	A3C	16-D LiDAR + goal distance and angle	**Continuous**: mean and standard deviation	**Sparse**	Employing a preliminary, non-expert policy to help the agent explore the environment.
2020	[81]	DDPG	40 × 1,440 LiDAR image + 2-d goal information	**Continuous**: linear and angular velocity	**Dense**: combination of sub-rewards	Policy can be transferred from simulation to real robot without the need for finetuning
2020	[82]	DDPG	20-D LiDAR + 1-D linear vel + 2-D goal information	**Continuous**: linear velocity + change in yaw	**Dense**: scaled distance difference	Deep RL algorithms are compared to a geometric controller and demonstrate goal reaching and obstacle avoidance
2020	[12]	PPO	Sequence of LiDAR observations + goal distance	**Continuous**: linear and angular velocity	**Dense**: scaled distance difference + MNR	The use of a Matching Network (MN) to shape reward
2020	[96]	DQN	LiDAR + previous action + pose estimate + map completeness percentage + timesteps till end of episode	**Discrete**: Forward, Back, Left, Right	**Dense**: Map Completeness Reward + Information Gain Reward	The use of SLAM to augment agent reward to speedup convergence
2020	[65]	DDQN	13-D LiDAR + 2-D goal information	**Discrete**: angular velocity 90, 45, 0, 45, 90	Dense: distance difference	The authors demonstrate that discrete space algorithms can outperfom continuous ones
2020	[94]	DDPG	100-D LiDAR + previous action + distance to goal	**Continuous**: linear and angular velocity	Dense: obstacle avoidance + target reaching + exploration	The use of the map's entropy to encourage exploration via the reward function
2020	[87]	DDPG	15-D LiDAR	**Continuous**: angular velocity of left and right wheel	**Dense**: scaled distance-difference + velocity reward	The authors start with a simple policy network and modify until smooth trajectories are obtained
2020	[95]	DDPG	2-D point cloud + agent's pose estimate + target position	**Continuous**: linear and angular velocity	**Dense**: obstacle avoidance + goal reaching + reward for keeping goal in FOV	The LiDAR data are encoded into a 2-D point-cloud representation. The use of an RGB-D camera to calculate FOV reward

Table 2: (continued)

Year	Ref	Algorithm	State	Action	Reward	Novelty
2020	[97]	DDPG	10-D range readings + goal information + agent's linear and angular velocity	**Continuous**: linear and angular velocity	**Dense**: scaled distance-difference	The actor-network of DDPG is modified to include a Spiking Neural Network (SNN)
2021	[86]	DQN & PPO	24-D LiDAR	**Discrete**: angular velocity of	**Dense**: obstacle avoidance + goal reaching	The fusion of obstacle avoidance and goal reaching using custom advantage function
2021	[90]	A2C	30-D LiDAR + goal information	**Discrete**: 7 possible linear velocities and 7 possible angular velocities	**Sparse**: 100 for reaching goal, −50 for collisions, 0 otherwise	The use of LSTM to remedy partial-observability & the transfer of policy to real-robot without domain adaptation
2021	[98]	SAC	36-D laser scans + agent velocity + goal information	**Continuous**: linear & angular velocity	**Dense**: scaled distance-difference + penalty for no movement	The agents laser readings are pre-processed into point representation
2021	[99]	DDPG	64-D LiDAR + goal information	**Continuous**: linear & angular velocity	**Dense**: step penalty + negative Euclidean distance to goal + negative angular speed + distance to closest obstacle	The use of AutoRL for parameter discovery & the combination of learning techniques with an SFM for social-navigation
2021	[91]	DDPG	20-D range readings + linear and altitude velocity + yaw differential + goal information	**Continuous**: linear velocity + yaw differential	**Sparse**: 100 for reaching goal, −10 for collisions	The authors demonstrate medium transition during navigation; showing the adaptiveness of RL for navigation
2022	[89]	DDPG	50-D range readings	**Continuous**: linear & angular velocity	**Dense**: angular velocity penalty + obstacle avoidance R + linear velocity R + exploration R	The use of an occupancy-grid reward to encourage exploration of the environment
2022	[100]	DQN	37-D laser scan + subgoal information + linear and angular velocity	**Discrete** 1 value output for linear velocity $\in$ {0, 0.05, 0.1, 0.15, 0.2}	**Dense**: scaled distance-difference + action reward + collision reward	The use of a hierarchy of stacked agents & the sepration of navigation into high and low-level policies
2022	[101]	DDPG	10-D range readings + angleDifference(goal) + goal information + yaw	**Continuous**: linear & angular velocity	**Dense**: distance penalty from goal and reward for moving away from starting point	The inclusion of LSTM layers to reduce partial observability & addition of noise to reduce training time and steps needed for convergence

Table 2: (continued)

Year	Ref	Algorithm	State	Action	Reward	Novelty
2022	[102]	TD3	LiDAR readings + distance to goal + previous and curent angular and linear velocity	**Continuous**: linear and angular velocity	**Dense**: linear velocity reward + angular velocity reward	The use of expert priors to bootstrap learning and extract reward function to use as baseline for agent's decisions
2022	[103]	DQN	24-D LiDAR + goal information + confidence of detected object + distance to nearest obstacle + angle to nearest obstacle	**Discrete**: 1 value for angular velocity $\in$ {-1.5, −0.75, 0, 0.75, 1.5}	**Dense**: distance-to-goal reward & angle-to-goal reward	The use of a pretrained object detector to locate goal and set targets & the projection of detected target from pixel to cartesian coordinates
2022	[92]	DDPG and SAC	20-D LiDAR + previous action + goal information	**Continous**: linear velocitiy & yaw differential	**Sparse**: 100 for goal reaching, −10 for collisions	The inclusion of LSTM layers into policy & the authors note that deep architectures deteriorate agent performance is deterministic case
2022	[93]	TD3	20-D LiDAR + previous action + goal information	**Continuous**: linear velocity, altitude velocity, and yaw	**Sparse**: 100 for reaching goal, −10 for collisions	The inclusion of RNN layers into agents' design enables navigation using less information
2023	[83]	DQN	24-D LiDAR + goal information	**Discrete**: angular velocity $\in$ {-1.5, −0.75, 0, 0.75, 1.5}	**Sparse**: 200 for goal reaching, −20 for collisions	The use of constant angular velocity results in a smaller action-space
2023	[84]	TD3	80-D LiDAR + waypoints	**Continuous**: linear and angular, subgoal coordinates velocity	**Dense**: collision penalty + distance to waypoint + safety reward	The use of a path-planning algorithm that is sampled from to train obstacle avoidance and goal reaching agents
2023	[19]	DDPG	LiDAR + linear velocity + angular velocity	**Continuous**: velocities of left and right wheel	**Dense**: scaled distance difference	The authors remove goal information from the state representation and use a Kalman filter to predict movement
2023	[85]	MPC + SQL	360-D LiDAR	**Continuous**: linear and angular velocity	**Dense**: running cost	The combination of model predictive control and reinforcement learning to speed up training and the use of LiDAR to detect obstacle shapes

Table 2: (continued)

Year	Ref	Algorithm	State	Action	Reward	Novelty
2025	[9]	CPO	LiDAR + polar coordinates of goal	**Continuous**: linear and angular velocity	**Dense**: distance reward + arrival reward + step penalty	The use of a hierarchical agent with a top-level agent utilizing LiDAR data to construct occupancy maps
2025	[1]	SAC	LiDAR	**Continuous**: linear and angular velocity	**Dense**: scaled distance-difference + time penalty	The development of a training method to enable robots of different sizes to contribute to learning the same policy

data as input may be infeasible when a dense reward mechanism is used. A sparse mechanism may overcome this limitation at the cost of increased training and convergence times. To illustrate, consider an agent using a sparse reward, the agent only receives a 1 when the goal is reached and 0 otherwise, until the agent reaches the goal for the first time, all state-action values will be 0. Table 2 provides a summary of recent approaches to mapless navigation that made use of LiDAR.

Image data in the state representation has also been used extensively to form the state representation of agents across multiple recent works. An onboard camera provides image data to the agent, which can be used for downstream tasks like object detection, image segmentation, and image classification. A natural formulation of the mapless navigation is to collect a large sample of data for pre-training certain aspects of an intelligent agent. The use of image data as input for decision-making systems can be attributed to the recent success of deep convolutional neural networks and, more recently, the development of the Vision Transformer (ViT) and other computer hardware advancements like faster processors [63]. RL has demonstrated the ability to learn the mapping between an input image and an action, as in [104]–[106]. The use of image data enables the specification of the agents goal as an image. The specification of the goal as an image as in [107]–[109] removes the need for localization of the goal or specification as a point as in [110].

The use of image data as input allows for the calculation of similarity-based reward mechanisms which reward the agent for having the goal in its FOV as in [95], [111] or detecting the goal from the current observation as in [106]. In their work, de Jesus et al. [112] demonstrate how RGB-D and RGD images can be utilized to alter the training of an agent in simulation; during training, the agent uses RGD-D, which is readily available in simulation, during evaluation, the agent uses only RGB data. Table 3 summarises recent approaches to mapless navigation that rely on image data as input for action selection.

The **combination of image and LiDAR** has also proven useful to speed up convergence and produce more robust agents capable of processing information from a variety of sources. The goal can now be expressed as a point [121], [122], an image [123] or no goal specification in the state space [124]. Table 4 provides a summary of recent approaches which combine LiDAR readings with image data to achieve navigation.

5.1 Limitations of the proposed study

The proposed study acknowledges several limitations that were not fully addressed in the review. First, while our review primarily focuses on reinforcement learning (RL) approaches for mapless navigation, we recognize the

Table 3: Summary of recent approaches utilizing image data.

Year	Ref	Algorithm	State	Action	Reward	Novelty
2019	[107]	A2C	174×174 current frame + image of goal + previous action + previous reward	**Discrete**: 1 predefined value $\in$ {forward, backwards, left, right, rotateLeft, rotateRight, tiltUp, tiltDown}	**Sparse**: 1 for goal reaching, 0 otherwise	The use of Auxiliary image-based tasks (depth map, segmentation, and goal segmentation) to improve learning and policy convergence
2020	[111]	A3C	RGB image frame + target information + scene prior context	**Discrete**: {moveAhead, rotateLeft, rotateRight, lookUp, lookDown and Done}	**Dense**: target-object visibility reward	The use of an object detector for construction of knowledge-graph & the use of Knowledge Graph to improve learning
2020	[113]	MLP Policy	4 RGB frames + image of target + previous action	**Discrete**: {moveForward, moveBack, moveLeft, rotate cw, rotate ccw, stop}	N/A	The use of goal prediction and next state prediction using expert trajectories
2020	[106]	A3C	RGB frame	**Discrete**: {moveAhead, moveBack, moveRight, moveLeft, lookUp, lookDown, done}	**Dense**: bounding-box reward	The use of a pretrained object detector to enhance reward & generation of context-grid using word2vec embeddings of target class
2020	[109]	CNN Policy	RGB or RGB-D image + target image	**Discrete**: {moveForward, moveBack, moveLeft, moveRight, rotateCCW, rotateCW, stop}	N/A	The authors demonstrate the learning of an image-based navigation agent trained using the Active Vision Dataset and capable of imagining next observations
2021	[114]	Gradient-based meta learning	$4 \times 224 \times 224$ RGB image + GloVe embedding of target object class + semantic knowledge-graph	**Discrete**: {moveAhead, turnLeft, turnRight, turnLeft, lookUp, lookDown, done}	**Sparse**: 5 for reaching goal, -0.1 for collisions, -0.01 otherwise	The use of semantic knowledge and meta-learning to improve policy & use of pretrained ResNet18 as feature extractor
2021	[104]	CNN-Policy	RGB image	**Discrete**: {goLeft, goRight, goStraight}	N/A	Active-vision dataset used to bootstrap training and speedup convergence of policy
2021	[115]	DDPG	64×64 segmented image + gait velocity + yaw of head joint + goal information	**Continuous**: *gait* velocity and yaw increments	**Dense**: target attention reward + distance reward + angle reward + movement penalty + obstacle avoidance R	Humanoid robot navigation by learning environment dynamics using an autoencoder for image data and for numeric data

Table 3: (continued)

Year	Ref	Algorithm	State	Action	Reward	Novelty
2021	[108]	A3C	Image observation + goal image	**Discrete**: {stepForward, stepBackward, turnRight, turnLeft}	**Sparse**: positive reward for reaching goal, reward ≤ 0 for timeout	The use of environment classification network produces behaviour according to environment
2022	[116]	A3C	RGB image + velocity + previous action + previous reward	**Discrete**: moveForward, turnLeft, turnRight, move forward and turn left/right	**Dense**: intrinsic novelty rewards	The use of multiple agents in parallel to speed up learning and policy convergence & the efficient reuse of trajectory data by incorporating previous reward into state
2022	[117]	SAC	State	**Continuous**: $a \in \Re^3$ – translational velocity, rotational velocity, and head rotation	**Dense**: speed reward + collision penalty + map reward	The use of 3-D convolution to encode past observations, actions, and rewards
2022	[105]	Dyna-Q	20×7 RGB image	**Discrete**: rotation angle $\in$ {-135, −90, −45, 0, 45, 90, 135}	**Sparse**: 0 for reaching goal, −0.1 otherwise	The demonstration of target-driven navigation from low-res images & the visualization of action preferences
2023	[118]	DQN	3 consecutive RGB-D images	**Discrete**: {turnLeft, turnRight, moveForward}	**Dense**: heading reward	The vanilla DQN is combined with Artificial Potential Field (APF)
2023	[119]	CNN Policy	96×96 input image	N/A	N/A	The use of a segmentation map with an overlayed path-planning algorithm & the use of image processing to augment input image to produce a local map & the use of A* for path-planning
2023	[110]	SAC	RGB $4 \times 160 \times 120$ RGB image + goal information	**Continuous**: linear and angular velocity	**Dense**: heuristic reward + action reward + goal reaching R + collision penalty	The use of a pre-trained ViT to aid goal reaching
2023	[120]	SAC	BEV image	**Continuous**: lateral position + heading angle + speed + acceleration	**Dense**: agent is rewarded for making progress towards goal and overtaking other cars	The authors propose the use of pre-trained actor and critic networks to leverage expert priors

Table 3: (continued)

Year	Ref	Algorithm	State	Action	Reward	Novelty
2025	[112]	SAC	RGB-D + RGB + robot position information + goal position information	**Continuous**: linear and angular velocity	**Dense**: scaled distance difference	A compact representation of RGB-D is learned in simulation where RGB-D is readily available. During evaluation only RGB images are utilized to predict actions.

Table 4: Summary of recent approaches utilizing combined data inputs.

Year	Ref	Algorithm	State	Action	Reward	Novelty
2019	[123]	DDPG	LiDAR + RGB image + goal image	Continuous: angular velocity for each wheel	Dense: weighted sum of linear and angular velocities	The decomposition of the navigation into two separately trained agents
2021	[124]	MaxPain	32×32 RGB image + 360-D LiDAR	**Discrete**: {moveLeft, moveRight, moveForward, rotateLeft, rotateRight}	**Sparse**: +1 for reaching goal and −0.1 for collisions	The authors propose a fusion of image and range data and a joint policy combination mechanism
2022	[121]	PPO	LiDAR + RGB image + goal information	**Continuous**: linear and angular velocity	**Dense**: termination R + position R + velocity R	The use of Curriculum Learning (CL) to improve robustness of policy and ease learning
2023	[122]	PPO	RGB image + range readings + goal information	**Continuous**: linear and angular velocity	**Dense**: encourage movement towards goal in horizontal and vertical axis	The agent predicts a depth-map from input using Depth Prediction Transformer (DPT)

potential advantages of hybrid methods that combine RL with traditional techniques such as SLAM or rule-based systems. Given the scope of our study, this focus may limit the applicability of our findings in scenarios where hybrid approaches could prove more effective. Secondly, our review mainly emphasizes LiDAR and image-based state representations, without considering other sensor modalities such as ultrasonic or radar. These alternative sensors may offer unique benefits in specific environments, and their exclusion represents a notable limitation.

Another limitation lies in our reliance on open-access literature sourced from selected databases. By excluding proprietary or unpublished research, there is a risk of introducing bias and omitting valuable insights that could enrich the findings. Additionally, the rapid pace of advancement in RL algorithms means that some of the methods reviewed may become outdated by the time of publication, further limiting the review's long-term relevance. Furthermore, our study lacks empirical validation of the surveyed approaches. Although we provide a thorough synthesis of the literature, we did not conduct comparative experiments to evaluate the performance of different RL techniques under standardized conditions. This absence of experimentation and result analysis makes it difficult to draw definitive conclusions about the superiority of one method over another.

While we identified challenges such as sparse rewards and sample inefficiency, we did not propose concrete solutions, as our aim was to provide a critical review rather than a comparative implementation. Nonetheless, these gaps present promising opportunities for future research. Finally, practical deployment considerations, including computational resource requirements, real-time processing constraints, and the robustness of RL policies in dynamic or unpredictable environments, were not discussed in depth. These limitations point to the

need for future studies that offer a more comprehensive perspective, incorporating empirical validation, diverse sensor integration, and practical deployment challenges.

6 Conclusion and future research directions

Computing devices have enabled the development of software for automation and scalability of repetitive tasks. When a software agent is required to interact with its environment, potentially changing the state of its environment through a set of actions, we enter the domain of robotics. Mobile robots are a subset of robots tasked with navigation in an environment. Defining the rules that govern an environment is infeasible; to solve this, model-free reinforcement learning allows for navigation agents to be trained through trial-and-error interaction with the environment. This is an exciting avenue of machine learning as it allows us to move away from the traditional modeling framework in which a phenomenon is characterized by a data set, to a learning paradigm in which the rules are defined and the model learns through interaction with the rules. In this work, we have reviewed the state-of-the-art w.r.t mobile robot navigation. We have reviewed the key components of a mobile robot, defined navigation mathematically, investigated the importance of the state and action space, how rewards can be designed effectively, and discussed how mapless navigation has grown in popularity and relevance. Our discussion is focused primarily on how reinforcement learning can be applied to a navigation task, but the fundamentals remain the same regardless of the task; as such, this review serves to introduce the reader to reinforcement learning from a goal-driven mapless navigation point of view. Based on the literature review.

- LiDAR data provides a cost-effective approach for realizing mapless navigation of an intelligent agent.
- Reward decomposition and enrichment improve policy convergence. The use of LiDAR allows for the construction of an obstacle avoidance reward. Image data is used for image segmentation, object detection for the creation of knowledge graphs, and image homography and perspective projection.
- Deep Q-Network (DQN) DQN variants, and actor-critic variants are the most popular choice for navigation of agents.
- Expert demonstrations provide an effective way of guiding an agents behaviour. Off-policy RL methods are used to pre-train certain aspects of an agent, like state enrichment or reward calculations.
- Model-based RL methods have been used to learn environment dynamics.
- To increase robustness of policy and speedup convergence, curriculums have been setup which increase the difficulty of the task gradually. Furthermore, domain randomisation is used to randomise starting points, object locations, goal locations, and object colours.
- Traditional planning methods in combination with image processing and DRL are used to provide interesting state and reward representations.
- Shortest Path Length (SPL), Success Rate (SR), Collision Rate (CR), and average reward are commonly used measures of gauging the performance of a navigation system. The Success Rate (SR) is defined as the fraction of episodes that end in success. Analysis of SR will reveal if an agent is reaching its goal. The Collision Rate (CR) is defined as the fraction of episodes that end in collision with obstacles. This gives insight into the agents obstacle avoidance capabilities. Analysis of an agent's reward over time may reveal how much reward is being received, but it does not provide an indication of success or failure at a given task. Furthermore, the Shortest Path Length (SPL) is not a good measure of performance when the shortest path is not ideal for an agent to take, i.e. the shortest path may contain more obstacles. To effectively gauge an agent's performance, several measures must be combined to get a holistic view of an agent's performance.

The study of mapless navigation is an ever-evolving landscape. Thus, there will always be room for improvement. This section provides possible promising avenues for future exploration.

- Autonomous Reward Function Design: The current study highlights the challenge of designing effective reward functions, especially in sparse reward scenarios. Future research could focus on developing methods for autonomous reward shaping, where the agent learns to construct its own reward function through interaction with the environment. Techniques like inverse reinforcement learning (IRL) or meta-learning

could be explored to enable agents to infer rewards from demonstrations or environmental feedback, reducing reliance on handcrafted rewards and improving adaptability.

– Multi-Agent Collaborative Navigation: While our review discusses hierarchical and stacked agents, there is potential for deeper exploration of collaborative multi-agent systems for mapless navigation. Future work could investigate how heterogeneous agents (e.g., drones, ground robots) with complementary sensors (LiDAR, cameras) can share learned policies or environmental representations in real-time. This could involve federated learning or communication-efficient reinforcement learning to enhance scalability and robustness in dynamic environments.
– Integration of Generative AI for Sim-to-Real Transfer: The current review notes the difficulty of transferring policies from simulation to real-world settings due to domain gaps. Future research could leverage generative AI models (e.g., diffusion models, GANs) to synthesize realistic training environments or augment

Table 5: Abbreviations used in this text.

Abbreviation	Meaning
CNN	Convolutional Neural Network
RL	Reinforcement Learning
ML	Machine Learning
AI	Artificial Intelligence
SR	Success Rate
CR	Collision Rate
SPL	Success weighted by Path Length
MDP	Markov Decision Process
DQN	Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
SAC	Soft Actor-critic
LiDAR	Light Detection and Ranging
RGB-D	Red Green Blue – Depth
RGB	Red Green Blue
UAV	Unmanned Aerial Vehicle
GPS	Global Positioning System
TD	Temporal Difference
SLAM	Simultaneous Localization and Mapping
SQL	Stacked Q-Learning
MPC	Model Predictive Control

Table 6: variables used in this text.

Symbol	Meaning
π	Policy
$\pi(s, a)$	Probability of select action a in state s
π_θ	Policy π parameterized by θ
$P(s_{t+1}\|s_t, a_t)$	Probability of being in state s_t, selecting a_t, and arriving at s_{t+1}
$v_\pi(s)$	The expected discounted sum of reward, starting in state s, and following π
γ	Discount factor
θ	Parameter matrix
α	Learning rate
τ	Agent trajectory

sensor data (e.g., LiDAR, RGB-D) with noise and variations. This would improve policy generalization and reduce the need for extensive real-world data collection, addressing challenges like sensor noise and environmental diversity.

Clearly, these directions align with the review's identified gaps (e.g., reward sparsity, scalability, sim-to-real transfer) and leverage emerging technologies to push the boundaries of mapless navigation. All abbreviations and variables used in this text are listed in Tables 5 and 6 along with their corresponding meaning.

Declaration of Conflicting Interests: The authors declare that there is no conflict of interest with regard to the publication of this paper.
Data Availability Statements: All data generated or analyzed during this study are included in this article.
Declaration of AI-assisted Technologies in Writing Process: During the preparation of this paper, the authors utilized AI-assisted tools such as Grammarly and QuillBot to enhance English language accuracy, including spelling, grammar, and punctuation. The content generated or corrected by these tools was thoroughly reviewed, revised, and edited by the authors to ensure accuracy and originality. The author accepts full responsibility for the originality of the final content of this study. All intellectual content and analysis remain the original work of the authors.
Funding Information: This research received no external funding.
Contribution Statement: V. Kok conducted the literature review, synthesized the findings, and drafted the manuscript. A. Ezugwu contributed to the conceptual framing, guided the critical evaluation of the sources, and provided editorial oversight by revising, writing, and redrafting several sections of the review. M. Olusanya guided evaluation of sources and guided organization of this work. All authors contributed to the development of the manuscript and approved its final version.

References

[1] W. Zhang, Y. Zhang, N. Liu, K. Ren, and G. Peng, "Dimension-variable mapless navigation with deep reinforcement learning," *IEEE ASME Trans. Mechatron.*, vol. 30, no. 6, 2025, https://doi.org/10.1109/TMECH.2025.3541797.
[2] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning: A brief survey," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, 2017, https://doi.org/10.1109/MSP.2017.2743240.
[3] V. Mnih *et al*., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, https://doi.org/10.1038/nature14236.
[4] M. Kuribayashi, K. Uehara, A. Wang, S. Morishima, and C. Asakawa, "WanderGuide: Indoor map-less robotic guide for exploration by blind people," arXiv preprint arXiv:2502.08906, 2025, https://doi.org/10.1145/3706598.3713788.
[5] H. Le, S. Saeedvand, and C. C. Hsu, "A comprehensive review of mobile robot navigation using deep reinforcement learning algorithms in crowded environments," *J. Intell. Rob. Syst.*, vol. 110, no. 4, p. 158, 2024, https://doi.org/10.1007/s10846-024-02198-w.
[6] X. Liu *et al*., "Enhancing navigation performance in unknown environments using spiking neural networks and reinforcement learning with asymptotic gradient method," *Complex Intell. Syst.*, vol. 11, no. 2, pp. 1–13, 2025, https://doi.org/10.1007/s40747-024-01777-6.
[7] J. C. Andersen, O. Ravn, and N. A. Andersen, "Autonomous rule-based robot navigation in orchards," *IFAC Proc. Vol.*, vol. 43, no. 16, pp. 43–48, 2010, https://doi.org/10.3182/20100906-3-it-2019.00010.
[8] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, Canberra, ACT, Australia, IEEE, 2020, December, pp. 737–744.
[9] J. Gao, X. Pang, Q. Liu, and Y. Li, "Hierarchical reinforcement learning for safe mapless navigation with congestion estimation," arXiv preprint arXiv:2503.12036, 2025, https://doi.org/10.1109/ICRA55743.2025.11128867.
[10] L. Ge, X. Zhou, Y. Li, and Y. Wang, "Deep reinforcement learning navigation via decision transformer in autonomous driving," *Front. Neurorob.*, vol. 18, 2024, Art. no. 1338189, https://doi.org/10.3389/fnbot.2024.1338189.
[11] A. Zhang, H. Sikchi, A. Zhang, and J. Biswas, "CREStE: Scalable mapless navigation with internet scale priors and counterfactual guidance," arXiv preprint arXiv:2503.03921, 2025, https://doi.org/10.15607/RSS.2025.XXI.136.
[12] Q. Zhang, M. Zhu, L. Zou, M. Li, and Y. Zhang, "Learning reward function with matching network for mapless navigation," *Sensors*, vol. 20, p. 3664, 2020, https://doi.org/10.3390/s20133664.
[13] M. Vecerik *et al*., "Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards," arXiv preprint arXiv:1707.08817, 2017.

[14] C. Wang, J. Wang, J. Wang, and X. Zhang, "Deep-reinforcement-learning-based autonomous UAV navigation with sparse rewards," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6180–6190, 2020, https://doi.org/10.1109/JIOT.2020.2973193.
[15] D. Andre and S. J. Russell, "State abstraction for programmable reinforcement learning agents," in *Aaai/iaai*, 2002, July, pp. 119–125.
[16] J. Liu, J. Guo, Z. Meng, and J. Xue, "ReVoLT: Relational reasoning and voronoi local graph planning for target-driven navigation," arXiv preprint arXiv:2301.02382, 2023.
[17] M. B. Alatise and G. P. Hancke, "A review on challenges of autonomous mobile robot and sensor fusion methods," *IEEE Access*, vol. 8, pp. 39830–39846, 2020, https://doi.org/10.1109/ACCESS.2020.2975643.
[18] F. Rubio, F. Valero, and C. Llopis-Albert, "A review of mobile robots: concepts, methods, theoretical framework, and applications," *Int. J. Adv. Rob. Syst.*, vol. 16, no. 2, 2019, Art. no. 1729881419839596, https://doi.org/10.1177/1729881419839596.
[19] H. Li, B. Luo, W. Song, and C. Yang, "Predictive hierarchical reinforcement learning for path-efficient mapless navigation with moving target," *Neural Netw.*, vol. 165, pp. 677–688, 2023, https://doi.org/10.1016/j.neunet.2023.06.007.
[20] G. A. Rummery and M. Niranjan, *On-line Q-learning Using Connectionist Systems*, vol. 37, Cambridge, University of Cambridge, Department of Engineering, 1994, p. 14.
[21] Y. Li, Q. Lyu, J. Yang, Y. Salam, and B. Wang, "Visual target-driven robot crowd navigation with limited FOV using self-attention enhanced deep reinforcement learning," *Sensors*, vol. 25, no. 3, p. 639, 2025, https://doi.org/10.3390/s25030639.
[22] W. Xiao, L. Yuan, T. Ran, L. He, J. Zhang, and J. Cui, "Multimodal fusion for autonomous navigation via deep reinforcement learning with sparse rewards and hindsight experience replay," *Displays*, vol. 78, 2023, Art. no. 102440, https://doi.org/10.1016/j.displa.2023.102440.
[23] K. Zhu and T. Zhang, "Deep reinforcement learning based mobile robot navigation: A review," *Tsinghua Sci. Technol.*, vol. 26, no. 5, pp. 674–691, 2021, https://doi.org/10.26599/TST.2021.9010012.
[24] P. Goel, S. I. Roumeliotis, and G. S. Sukhatme, "Robust localization using relative and absolute position estimates," in *Proceedings 1999 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human and Environment Friendly Robots with High Intelligence and Emotional Quotients (Cat. No. 99CH36289)*, vol. 2, Kyongju, South Korea, IEEE, 1999, October, pp. 1134–1140.
[25] S. Thrun, "Finding landmarks for mobile robot navigation," in *Proceedings. 1998 IEEE International Conference on Robotics and Automation (Cat. No. 98CH36146)*, vol. 2, Leuven, Belgium, IEEE, 1998, May, pp. 958–963.
[26] D. S. Chaplot, D. P. Gandhi, A. Gupta, and R. R. Salakhutdinov, "Object goal navigation using goal-oriented semantic exploration," *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 4247–4258, 2020.
[27] A. Chatterjee, K. Pulasinghe, K. Watanabe, and K. Izumi, "A particle-swarm-optimized fuzzy-neural network for voice-controlled robot systems," *IEEE Trans. Ind. Electron.*, vol. 52, no. 6, pp. 1478–1489, 2005, https://doi.org/10.1109/TIE.2005.858737.
[28] H. Sowerby, Z. Zhou, and M. L. Littman, "Designing rewards for fast learning," arXiv preprint arXiv:2205.15400, 2022.
[29] J. Eschmann, "Reward function design in reinforcement learning," *Reinforc. Learn. Algorithm. Anal. Appl.*, vol. 833, pp. 25–33, 2021, https://doi.org/10.1007/978-3-030-41188-6_3.
[30] R. Devidze, G. Radanovic, P. Kamalaruban, and A. Singla, "Explicable reward design for reinforcement learning agents," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 20118–20131, 2021.
[31] M. N. Al-Hamadani, M. A. Fadhel, L. Alzubaidi, and B. Harangi, "Reinforcement learning algorithms and applications in healthcare and robotics: A comprehensive and systematic review," *Sensors*, vol. 24, no. 8, p. 2461, 2024, https://doi.org/10.3390/s24082461.
[32] S. Gattu, "Autonomous navigation and obstacle avoidance using self-guided and self-regularized actor-critic," in *Proceedings of the 8th International Conference on Robotics and Artificial Intelligence*, Singapore, Association for Computing Machinery (ACM), 2022, November, pp. 52–58.
[33] H. Jeong, H. Hassani, M. Morari, D. D. Lee, and G. J. Pappas, "Deep reinforcement learning for active target tracking," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, Xi'an, China, IEEE, 2021, pp. 1825–1831.
[34] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, London, UK, IEEE, 2023, May, pp. 10608–10615.
[35] F. Gul, W. Rahiman, and S. S. Nazli Alhady, "A comprehensive study for robot navigation techniques," *Cogent Eng.*, vol. 6, no. 1, 2019, Art. no. 1632046, https://doi.org/10.1080/23311916.2019.1632046.
[36] H. Lee and J. Jeong, "Mobile robot path optimization technique based on reinforcement learning algorithm in warehouse environment," *Appl. Sci.*, vol. 11, no. 3, p. 1209, 2021, https://doi.org/10.3390/app11031209.
[37] A. D. J. Plasencia-Salgueiro, "Deep reinforcement learning for autonomous mobile robot navigation," in *Artificial Intelligence for Robotics and Autonomous Systems Applications*, Cham, Springer International Publishing, 2023, pp. 195–237.
[38] G. S. Tewolde, "Sensor and network technology for intelligent transportation systems," in *2012 IEEE International Conference on Electro/Information Technology*, Indianapolis, IN, IEEE, 2012, May, pp. 1–7.
[39] U. Nehmzow and C. Owen, "Robot navigation in the real world: Experiments with Manchester's FortyTwo in unmodified, large environments," *Robot. Autonom. Syst.*, vol. 33, no. 4, pp. 223–242, 2000, https://doi.org/10.1016/s0921-8890(00)00098-1.
[40] M. Aizat, A. Azmin, and W. Rahiman, "A survey on navigation approaches for automated guided vehicle robots in dynamic surrounding," *IEEE Access*, vol. 11, pp. 33934–33955, 2023, https://doi.org/10.1109/ACCESS.2023.3263734.
[41] Y. H. Kim, H. J. Kim, J. H. Lee, E. J. Kim, and J. W. Song, "Sequential batch fusion magnetic anomaly navigation for a low-cost indoor mobile robot," *Measurement*, vol. 213, 2023, Art. no. 112706, https://doi.org/10.1016/j.measurement.2023.112706.
[42] S. Im, S. Kim, J. Yun, and J. Nam, "Robot-aided magnetic navigation system for wireless capsule manipulation," *Micromachines*, vol. 14, no. 2, p. 269, 2023, https://doi.org/10.3390/mi14020269.

[43] A. Pore *et al*., "Autonomous navigation for robot-assisted intraluminal and endovascular procedures: A systematic review," *IEEE Trans. Robot.*, vol. 39, no. 6, 2023, https://doi.org/10.1109/TRO.2023.3269384.
[44] G. Loianno *et al*., "Localization, grasping, and transportation of magnetic objects by a team of mavs in challenging desert-like environments," *IEEE Rob. Autom. Lett.*, vol. 3, no. 3, pp. 1576–1583, 2018, https://doi.org/10.1109/LRA.2018.2800121.
[45] M. Z. Baharuddin, I. Z. Abidin, S. S. K. Mohideen, Y. K. Siah, and J. T. T. Chuan, "Analysis of line sensor configuration for the advanced line follower robot," University Tenaga Nasional, 2005.
[46] M. Engin and D. Engin, "Path planning of line follower robot," in *2012 5th European DSP Education and Research Conference (EDERC)*, Amsterdam, Netherlands, IEEE, 2012, September, pp. 1–5.
[47] A. Siddiqui, A. Basker, M. Dias, K. Chavan, and N. Avhad, "Line following cart (LFC) for office applications," in *2023 5th Biennial International Conference on Nascent Technologies in Engineering (ICNTE)*, Navi Mumbai, India, IEEE, 2023, January, pp. 1–6.
[48] J. Yu *et al*., "Study of convolutional neural network-based semantic segmentation methods on edge intelligence devices for field agricultural robot navigation line extraction," *Comput. Electron. Agric.*, vol. 209, 2023, Art. no. 107811, https://doi.org/10.1016/j.compag.2023.107811.
[49] R. Farkh and K. Aljaloud, "Vision navigation based PID control for line tracking robot," *Intell. Autom. Soft Comput.*, vol. 35, no. 1, pp. 901–911, 2023, https://doi.org/10.32604/iasc.2023.027614.
[50] M. Y. Arafat, M. M. Alam, and S. Moh, "Vision-based navigation techniques for unmanned aerial vehicles: Review and challenges," *Drones*, vol. 7, no. 2, p. 89, 2023, https://doi.org/10.3390/drones7020089.
[51] Y. Bai, B. Zhang, N. Xu, J. Zhou, J. Shi, and Z. Diao, "Vision-based navigation and guidance for agricultural autonomous vehicles and robots: A review," *Comput. Electron. Agric.*, vol. 205, 2023, Art. no. 107584, https://doi.org/10.1016/j.compag.2022.107584.
[52] S. Yan, J. Wang, Z. Wu, M. Tan, and J. Yu, "Autonomous vision-based navigation and stability augmentation control of a biomimetic robotic hammerhead shark," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, 2023, https://doi.org/10.1109/TASE.2023.3278740.
[53] S. Tavasoli, X. Pan, and T. Y. Yang, "Real-time autonomous indoor navigation and vision-based damage assessment of reinforced concrete structures using low-cost nano aerial vehicles," *J. Build. Eng.*, vol. 68, 2023, Art. no. 106193, https://doi.org/10.1016/j.jobe.2023.106193.
[54] S. Kumar, M. Majeedullah, and A. B. Buriro, "Autonomous navigation and real time mapping using ultrasonic sensors in NAO humanoid robot," *Eng. Technol. Appl. Sci. Research*, vol. 12, no. 5, pp. 9102–9107, 2022, https://doi.org/10.48084/etasr.5180.
[55] T. R. Anusha, S. S. Kumar, and S. Panday, "ROS based obstacle detection robot using ultrasonic sensor and FMCW RADAR," in *2022 3rd International Conference on Electronics and Sustainable Communication Systems (ICESC)*, Coimbatore, India, IEEE, 2022, August, pp. 372–378.
[56] I. Ohya, A. Kosaka, and A. Kak, "Vision-based navigation by a mobile robot with obstacle avoidance using single-camera vision and ultrasonic sensing," *IEEE Trans. Robot. Autom.*, vol. 14, no. 6, pp. 969–978, 1998, https://doi.org/10.1109/70.736780.
[57] S. Yuan, J. Wu, F. Luan, L. Zhang, and J. Lv, "Improvement of strong tracking UKF-SLAM approach using three-position ultrasonic detection," *Robot. Autonom. Syst.*, vol. 159, 2023, Art. no. 104305, https://doi.org/10.1016/j.robot.2022.104305.
[58] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based SLAM," *IEEE Intell. Transport. Syst. Mag.*, vol. 2, no. 4, pp. 31–43, 2010, https://doi.org/10.1109/MITS.2010.939925.
[59] O. S. Ekundayo and A. E. Ezugwu, "Deep learning: Historical overview from inception to actualization, models, applications and future trends," *Appl. Soft Comput.*, vol. 181, no. 113378, 2025, https://doi.org/10.1016/j.asoc.2025.113378.
[60] J. Wang, S. Elfwing, and E. Uchibe, "Deep reinforcement learning by parallelizing reward and punishment using the maxpain architecture," in *2018 Joint IEEE 8th International Conference on Development and Learning and Epigenetic Robotics (ICDL-EpiRob)*, Tokyo, Japan, IEEE, 2018, September, pp. 175–180.
[61] L. Graesser and W. L. Keng, *Foundations of Deep Reinforcement Learning*, Columbus, Ohio, Addison-Wesley Professional, 2019.
[62] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, Cambridge, MIT Press, 2018.
[63] S. Cebollada, L. Payá, M. Flores, A. Peidró, and O. Reinoso, "A state-of-the-art review on mobile robotics tasks using artificial intelligence and visual data," *Expert Syst. Appl.*, vol. 167, 2021, Art. no. 114195, https://doi.org/10.1016/j.eswa.2020.114195.
[64] Q. Wang, Q. Lv, H. Wei, and P. Zhang, "A deep reinforcement learning based mapless navigation algorithm using continuous actions," in *2019 International Conference on Robots and Intelligent System (ICRIS)*, Haikou, China, IEEE, 2019, June, pp. 63–68.
[65] E. Marchesini and A. Farinelli, "Discrete deep reinforcement learning for mapless navigation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, Paris, France (Virtual), IEEE, 2020, May, pp. 10688–10694.
[66] C. Y. Tsai, H. Nisar, and Y. C. Hu, "Mapless LiDAR navigation control of wheeled mobile robots based on deep imitation learning," *IEEE Access*, vol. 9, pp. 117527–117541, 2021, https://doi.org/10.1109/ACCESS.2021.3107041.
[67] L. Dong, Z. He, C. Song, and C. Sun, "A review of mobile robot motion planning methods: From classical motion planning workflows to reinforcement learning-based architectures," *J. Syst. Eng. Electron.*, vol. 34, no. 2, pp. 439–459, 2023, https://doi.org/10.23919/JSEE.2023.000051.
[68] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *International Conference on Machine Learning*, Beijing, China, Pmlr, 2014, January, pp. 387–395.
[69] D. Muse and S. Wermter, "Actor-critic learning for platform-independent robot navigation," *Cogn Comput.*, vol. 1, pp. 203–220, 2009, https://doi.org/10.1007/s12559-009-9021-z.
[70] L. Xie, *Reinforcement Learning Based Mapless Robot Navigation*, (Doctoral dissertation, University of Oxford), 2019.

[71] M. Han, L. Zhang, J. Wang, and W. Pan, "Actor-critic reinforcement learning for control with stability guarantee," *IEEE Rob. Autom. Lett.*, vol. 5, no. 4, pp. 6217–6224, 2020, https://doi.org/10.1109/LRA.2020.3011351.
[72] T. P. Lillicrap *et al*., "Continuous control with deep reinforcement learning," arXiv preprint arXiv:1509.02971, 2015.
[73] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International Conference on Machine Learning*, Stockholm, Sweden, PMLR, 2018, July, pp. 1587–1596.
[74] J. C. de Jesus, V. A. Kich, A. H. Kolling, R. B. Grando, M. A. D. S. L. Cuadros, and D. F. T. Gamarra, "Soft actor-critic for navigation of mobile robots," *J. Intell. Rob. Syst.*, vol. 102, no. 2, p. 31, 2021, https://doi.org/10.1007/s10846-021-01367-5.
[75] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning*, Stockholm, Sweden, Pmlr, 2018, July, pp. 1861–1870.
[76] D. Moher *et al*., "Preferred reporting items for systematic review and meta-analysis protocols (PRISMA-P) 2015 statement," *Syst. Rev.*, vol. 4, no. 1, p. 1, 2015, https://doi.org/10.1186/2046-4053-4-1.
[77] R. Möller, A. Furnari, S. Battiato, A. Härmä, and G. M. Farinella, "A survey on human-aware robot navigation," *Robot. Autonom. Syst.*, vol. 145, 2021, Art. no. 103837, https://doi.org/10.1016/j.robot.2021.103837.
[78] Y. Chang, Y. Cheng, U. Manzoor, and J. Murray, "A review of UAV autonomous navigation in GPS-denied environments," *Robot. Autonom. Syst.*, 2023, Art. no. 104533, https://doi.org/10.1016/j.robot.2023.104533.
[79] F. AlMahamid and K. Grolinger, "Autonomous unmanned aerial vehicle navigation using reinforcement learning: A systematic review," *Eng. Appl. Artif. Intell.*, vol. 115, 2022, Art. no. 105321, https://doi.org/10.1016/j.engappai.2022.105321.
[80] T. Yang *et al*., "3D ToF LiDAR in mobile robotics: A review," arXiv preprint arXiv:2202.11025, 2022.
[81] J. Jin, N. M. Nguyen, N. Sakib, D. Graves, H. Yao, and M. Jagersand, "Mapless navigation among dynamics with social-safety-awareness: A reinforcement learning approach from 2d laser scans," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, Paris, France (Virtual), IEEE, 2020, May, pp. 6979–6985.
[82] R. B. Grando, J. C. de Jesus, and P. L. Drews-Jr, "Deep reinforcement learning for mapless navigation of unmanned aerial vehicles," in *2020 Latin American Robotics Symposium (LARS), 2020 Brazilian Symposium on Robotics (SBR) and 2020 Workshop on Robotics in Education (WRE)*, Natal, Brazil (Virtual), IEEE, 2020, November, pp. 1–6.
[83] L. D. de Moraes *et al*., "Double deep reinforcement learning techniques for low dimensional sensing mapless navigation of terrestrial Mobile robots," arXiv preprint arXiv:2301.11173, 2023, https://doi.org/10.1007/978-3-031-35507-3_16.
[84] J. de Heuvel, W. Shi, X. Zeng, and M. Bennewitz, "Subgoal-driven navigation in dynamic environments using attention-based deep reinforcement learning," arXiv preprint arXiv:2303.01443, 2023, https://doi.org/10.1109/ICAR58858.2023.10406349.
[85] D. Dobriborsci, R. Zashchitin, M. Kakanov, W. Aumer, and P. Osinenko, "Predictive reinforcement learning: Map-less navigation method for mobile robot," *J. Intell. Manuf.*, vol. 35, no. 8, pp. 4217–4232, 2024, https://doi.org/10.1007/s10845-023-02197-y.
[86] J. Li, M. Ran, H. Wang, and L. Xie, "A behaviour-based mobile robot navigation method with deep reinforcement learning," *Unmanned Syst.*, vol. 9, no. 03, pp. 201–209, 2021.
[87] W. Brink, "Learning fine-grained control for mapless navigation," in *2020 International SAUPEC/RobMech/PRASA Conference*, Cape Town, South Africa, IEEE, 2020, January, pp. 1–6.
[88] W. Chen, S. Zhou, Z. Pan, H. Zheng, and Y. Liu, "Mapless collaborative navigation for a multi-robot system based on the deep reinforcement learning," *Appl. Sci.*, vol. 9, no. 20, p. 4198, 2019, https://doi.org/10.3390/app9204198.
[89] A. Kamalova, S. G. Lee, and S. H. Kwon, "Occupancy reward-driven exploration with deep reinforcement learning for Mobile robot system," *Appl. Sci.*, vol. 12, no. 18, p. 9249, 2022, https://doi.org/10.3390/app12189249.
[90] M. Dobrevski and D. Skočaj, "Deep reinforcement learning for map-less goal-driven robot navigation," *Int. J. Adv. Rob. Syst.*, vol. 18, no. 1, 2021, Art. no. 1729881421992621, https://doi.org/10.1177/1729881421992621.
[91] R. B. Grando *et al*., "Deep reinforcement learning for mapless navigation of a hybrid aerial underwater vehicle with medium transition," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, Xi'an, China, IEEE, 2021, May, pp. 1088–1094.
[92] R. B. Grando *et al*., "Deterministic and stochastic analysis of deep reinforcement learning for low dimensional sensing-based navigation of Mobile robots," in *2022 Latin American Robotics Symposium (LARS), 2022 Brazilian Symposium on Robotics (SBR), and 2022 Workshop on Robotics in Education (WRE)*, São Bernardo do Campo, SP, Brazil, IEEE, 2022, October, pp. 193–198.
[93] R. B. Grando, J. C. de Jesus, V. A. Kich, A. H. Kolling, and P. L. J. Drews-Jr, "Double critic deep reinforcement learning for mapless 3d navigation of unmanned aerial vehicles," *J. Intell. Rob. Syst.*, vol. 104, no. 2, p. 29, 2022, https://doi.org/10.1007/s10846-021-01568-y.
[94] M. Botteghi, M. Khaled, B. Sirmacek, and M. Poel, "Entropy-based exploration for mobile robot navigation: A learning-based approach," in *Planning and Robotics Workshop*, PlanRob, 2020.
[95] F. Leiva and J. Ruiz-del-Solar, "Robust rl-based map-less local planning: Using 2d point clouds as observations," *IEEE Rob. Autom. Lett.*, vol. 5, no. 4, pp. 5787–5794, 2020, https://doi.org/10.1109/LRA.2020.3010732.
[96] N. Botteghi, B. Sirmacek, R. Schulte, M. Poel, and C. Brune, "Reinforcement learning helps slam: Learning to build maps. International archives of the photogrammetry," *Rem. Sens. Spatial Inf. Sci.*, vol. 43, 2020, https://doi.org/10.5194/isprs-archives-XLIII-B4-2020-329-2020.
[97] G. Tang, N. Kumar, and K. P. Michmizos, "Reinforcement co-learning of deep and spiking neural networks for energy-efficient mapless navigation with neuromorphic hardware," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Las Vegas, NV, USA, IEEE, 2020, October, pp. 6090–6097.
[98] W. Zhang, N. Liu, and Y. Zhang, "Learn to navigate maplessly with varied LiDAR configurations: A support point-based approach," *IEEE Rob. Autom. Lett.*, vol. 6, no. 2, pp. 1918–1925, 2021, https://doi.org/10.1109/LRA.2021.3061305.

[99] Ó. Gil, A. Garrell, and A. Sanfeliu, "Social robot navigation tasks: Combining machine learning techniques and social force model," *Sensors*, vol. 21, no. 21, p. 7087, 2021, https://doi.org/10.3390/s21217087.
[100] W. Zhu and M. Hayashibe, "A hierarchical deep reinforcement learning framework with high efficiency and generalization for fast and safe navigation," *IEEE Trans. Ind. Electron.*, vol. 70, no. 5, pp. 4962–4971, 2022, https://doi.org/10.1109/TIE.2022.3190850.
[101] H. Gong, P. Wang, C. Ni, and N. Cheng, "Efficient path planning for mobile robot based on deep deterministic policy gradient," *Sensors*, vol. 22, no. 9, p. 3579, 2022, https://doi.org/10.3390/s22093579.
[102] R. Cimurs and E. A. Merchán-Cruz, "Leveraging expert demonstration features for deep reinforcement learning in floor cleaning robot navigation," *Sensors*, vol. 22, no. 20, p. 7750, 2022, https://doi.org/10.3390/s22207750.
[103] M. F. R. Lee and S. H. Yusuf, "Mobile robot navigation using deep reinforcement learning," *Processes*, vol. 10, no. 12, p. 2748, 2022, https://doi.org/10.3390/pr10122748.
[104] T. Ran, L. Yuan, and J. B. Zhang, "Scene perception based visual navigation of mobile robot in indoor environment," *ISA Trans.*, vol. 109, pp. 389–400, 2021, https://doi.org/10.1016/j.isatra.2020.10.023.
[105] P. Blum, P. Crowley, and G. Lykotrafitis, "Vision-based navigation and obstacle avoidance via deep reinforcement learning," arXiv preprint arXiv:2211.05243, 2022.
[106] R. Druon, Y. Yoshiyasu, A. Kanezaki, and A. Watt, "Visual object search by learning spatial context," *IEEE Rob. Autom. Lett.*, vol. 5, no. 2, pp. 1279–1286, 2020, https://doi.org/10.1109/LRA.2020.2967677.
[107] J. Kulhánek, E. Derner, T. De Bruin, and R. Babuška, "Vision-based navigation using deep reinforcement learning," in *2019 European Conference on Mobile Robots (ECMR)*, Prague, Czech Republic, IEEE, 2019, September, pp. 1–8.
[108] B. Xihan, O. Mendez, and S. Hadfield, "Robot in a China shop: Using reinforcement learning for location-specific navigation behaviour," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, Xi'an, China, IEEE, 2021, May, pp. 5959–5965.
[109] Q. Wu, D. Manocha, J. Wang, and K. Xu, "Neonav: Improving the generalization of visual navigation via generating next expected observations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, New York, NY, USA, AAAI Press, 2020, April, pp. 10001–10008.
[110] W. Huang, Y. Zhou, X. He, and C. Lv, "Goal-guided transformer-enabled reinforcement learning for efficient autonomous navigation," arXiv preprint arXiv:2301.00362, 2023, https://doi.org/10.1109/TITS.2023.3312453.
[111] Y. Qiu, A. Pal, and H. I. Christensen, "Target driven visual navigation exploiting object relationships," arXiv preprint arXiv:2003.06749, vol. 2, no. 7, 2020.
[112] J. Costa de Jesus, V. A. Kich, A. H. Kolling, R. B. Grando, R. da Silva Guerra, and P. L. J. Drews-Jr, "Image-based mapless navigation of a hybrid aerial-underwater vehicle using prioritized deep reinforcement learning," *J. Intell. Rob. Syst.*, vol. 111, no. 1, p. 27, 2025, https://doi.org/10.1007/s10846-024-02206-z.
[113] Q. Wu, X. Gong, K. Xu, D. Manocha, J. Dong, and J. Wang, "Towards target-driven visual navigation in indoor scenes via generative imitation learning," *IEEE Rob. Autom. Lett.*, vol. 6, no. 1, pp. 175–182, 2020, https://doi.org/10.1109/LRA.2020.3036597.
[114] F. Li, C. Guo, B. Luo, and H. Zhang, "Multi goals and multi scenes visual mapless navigation in indoor using meta-learning and scene priors," *Neurocomputing*, vol. 449, pp. 368–377, 2021, https://doi.org/10.1016/j.neucom.2021.03.084.
[115] A. Brandenburger, D. Rodriguez, and S. Behnke, "Mapless humanoid navigation using learned latent dynamics," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Prague, Czech Republic (Hybrid), IEEE, 2021, September, pp. 1555–1561.
[116] X. Ruan, P. Li, X. Zhu, and P. Liu, "A target-driven visual navigation method based on intrinsic motivation exploration and space topological cognition," *Sci. Rep.*, vol. 12, no. 1, p. 3462, 2022, https://doi.org/10.1038/s41598-022-07264-7.
[117] H. Xue, R. Song, J. Petzold, B. Hein, H. Hamann, and E. Rueckert, "End-to-end deep reinforcement learning for first-person pedestrian visual navigation in urban environments," in *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, Ginowan, Japan, IEEE, 2022, November, pp. 350–357.
[118] A. Sivaranjani and B. Vinod, "Artificial potential field incorporated Deep-Q-Network algorithm for Mobile robot path prediction," *Intell. Autom. Soft Comput.*, vol. 35, no. 1, pp. 1135–1135, 2023, https://doi.org/10.32604/iasc.2023.028126.
[119] T. V. Dang and N. T. Bui, "Multi-scale fully convolutional network-based semantic segmentation for Mobile robot navigation," *Electronics*, vol. 12, no. 3, p. 533, 2023, https://doi.org/10.3390/electronics12030533.
[120] L. Wang *et al*., "Efficient reinforcement learning for autonomous driving with parameterized skills and priors," arXiv preprint arXiv: 2305.04412, 2023, https://doi.org/10.15607/RSS.2023.XIX.102.
[121] N. Wang, Y. Wang, Y. Zhao, Y. Wang, and Z. Li, "Sim-to-Real: Mapless navigation for USVs using deep reinforcement learning," *J. Mar. Sci. Eng.*, vol. 10, no. 7, p. 895, 2022, https://doi.org/10.3390/jmse10070895.
[122] P. Yang, H. Liu, M. Roznere, and A. Q. Li, "Monocular camera and single-beam sonar-based underwater collision-free navigation with domain randomization," in *Robotics Research*, Cham, Springer Nature Switzerland, 2023, pp. 85–101.
[123] A. Wahid, A. Toshev, M. Fiser, and T. W. E. Lee, "Long range neural navigation policies for the real world," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Macau, China, IEEE, 2019, November, pp. 82–89.
[124] J. Wang, S. Elfwing, and E. Uchibe, "Modular deep reinforcement learning from reward and punishment for robot navigation," *Neural Netw.*, vol. 135, pp. 115–126, 2021, https://doi.org/10.1016/j.neunet.2020.12.001.